00 Contents

Every page of the note. The tab numbers match the boxes on the right edge.

START
CONTENTS

00 Start

Find the pattern	2
How to answer	3
Reading the question	4
Past-question lookup	5

B Basics

Design pattern basics	6
Drawing UML in the exam	7

01 Singleton

Singleton	8
Singleton: write	9
Singleton: past questions answered	10
Singleton: more answers	11
Singleton: Multiton, OOP and SOLID	12

02 Factory

Factory Method	13
Factory Method: write	14
Factory Method: past questions answered	15
Factory Method: more answers	16

03 Facade

Facade	17
Facade: write	18
Facade: answers and challenges	19

04 Adapter

Adapter	20
Adapter: write	21
Adapter: past questions answered	22

05 Proxy

Proxy	23
Proxy: write	24
Proxy: past questions answered	25

06 Decorator

Decorator	26
Decorator: write	27
Decorator: past questions answered	28

07 Template

Template Method	29
Template Method: write	30
Template Method: salary questions	31
Template Method: more answers	32

08 Chain

Chain of Responsibility	33
Chain of Responsibility: write	34
Chain: the spam filter question	35
Chain: validation checks and more	36

09 State

State	37
State: write	38
State: smart parking	39
State: kiosk and interfaces	40

10 SOLID

SOLID principles	41
SOLID: L, I, D	42

11 Micro

Microservices reference	43
Microservices: the other parts	44

12 Bank

Answer bank	45
Answer bank: other patterns	46

13 Worked

Worked answer: 2025 Q4	47
Worked answer: 2025 Q1	48

Page layout for every pattern: Understand (Spot it, the idea, the parts, comparisons, mistakes) → **Write** (W1 name and reasons, W2 diagram, W3 Java) → **Answers** (past questions, W4 extras, W5 challenges) → **More examples** and **OOP & SOLID** where added.

Find the pattern

00 Start here for every question. The words in the question tell you which pattern they want.

START
READ THIS FIRST

STEP 1
Underline the problem words
"optional add-ons", "only one", "states", "in order", "can't modify"...

STEP 2
Find them in the table
Each phrase points to one pattern and its tab on the right edge.

STEP 3
Open the tab, check "Spot it"
The first page of each pattern lists more phrases. If they match, you're right.

STEP 4
Write from the W blocks
W1, W2, W3... are in the order you write. Swap in the names from the question.

IF THE QUESTION SAYS SOMETHING LIKE...	IT WANTS	TAB
"only one instance", "a single global tax calculator / logger / ID generator", "only one user can run it", "a new object is created every time and causes duplicates"	**Singleton** creational	**01**
"a maximum of N": five batsmen, three printers, N gates	**Multiton** (extended Singleton) creational	**01**
"create different types of X depending on the user's request", "too many if-else blocks to create objects", "extensible, product-based framework"	**Factory Method** creational	**02**
"one simple interface that internally calls login, inventory, payment...", "combine air tickets, hotels and taxis", "hide the complexity"	**Facade** structural	**03**
"legacy", "supplied by an external vendor", "can't modify", "the new device has a different method", "third-party services with different method signatures"	**Adapter** structural	**04**
"load only a couple of pages", "users should not access it directly", "restricted archive", "permissions", "authorization options", "cache"	**Proxy** structural	**05**
"optional add-ons / extras / toppings", "without creating subclasses for every combination", "price depends on the base and the selected additions"	**Decorator** structural	**06**
"all types follow the same overall process, but step X is different", "standardise the workflow", "every level must follow the same procedure"	**Template Method** behavioural	**07**
"checks that must happen in order and be easy to extend", "levels of support / approval", "sub-filters", "different priorities", "exception handling"	**Chain of Responsibility** behavioural	**08**
"operates in different states", "transitions to", "behaviour depends on its current state", "draw the state diagram"	**State** behavioural	**09**
"which design principle...", "explain how this follows Open/Closed", "depend on abstractions"	**SOLID**	**10**
"microservices", "scalability, fault isolation and continuous deployment", "data consistency", "security"	**Microservices kit** architecture question	**11**
"High Availability", "Disaster Recovery", "configurability", "use of interfaces / abstract classes", a pattern you weren't taught (Strategy, Observer...)	**Answer bank**	**12**

00 How to answer

The same answer shape works for almost every part of every question.

THE PAPER
4 questions × 25
Answer all four. 2½ hours. Open book.

YOUR CLOCK
35 min each
About 1½ minutes per mark. Keep 10 minutes spare.

BEFORE YOU START
Write assumptions
The paper asks for them. One or two lines at the top.

"Which design pattern would you use? Give reasons."

the most common part

1. **Name it and its group.** "I would use the Decorator pattern, which is a structural pattern."
2. **Say what it does in one line.** Each pattern's W1 has this sentence ready.
3. **Map the scenario onto the pattern's parts.** "Coffee is the component, Milk and Sugar are decorators." Most of the marks are here.
4. **Give 2 or 3 reasons tied to the scenario.** Not general praise. Say what problem it removes here.
5. **Draw something if it's worth 4 marks or more.** A small class diagram is enough.
6. **Add code only if they ask for it.** Interface first, then the class, then 3 lines of client code.

How much to write

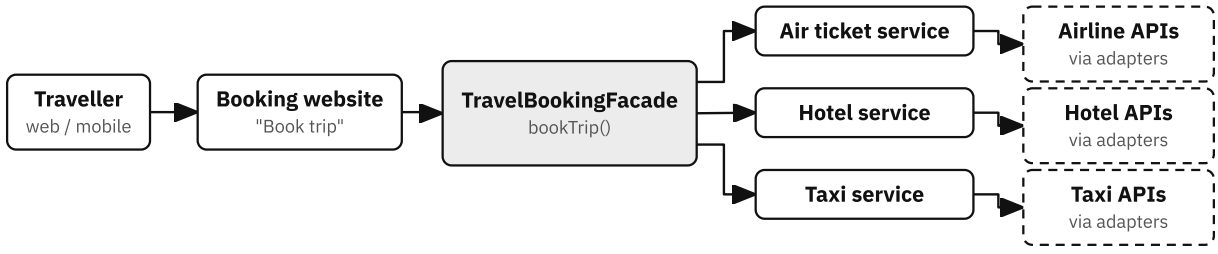
MARKS	WRITE
2–3	Pattern name and one solid reason.
4–5	Name, mapping, two or three reasons. A small sketch helps.
5–8 (diagram)	Full class diagram with the correct arrows, stereotypes and + / − / # signs.
3–6 (code)	Exactly what they asked for, using their method names.
"Challenges"	Each challenge paired with how you would solve it. Each W5 block has these pairs.

Assumptions you can reuse

- The system is written in Java (or a similar OO language).
- Prices and rates are read from configuration, not hard-coded.
- Only the classes needed to show the pattern are drawn. Getters and setters are left out.
- Third-party systems can't be changed by us.
- The system runs on several servers behind a load balancer.

"Draw a diagram to depict the scenario"

this is NOT a class diagram. Draw the real things and how they connect.



Example from 2022 Q1 (travel booking). Use boxes for users, systems and services, and arrows for requests. Label the arrows. Then show where your pattern sits (here: Facade in the middle, Adapters at the edge).

00 Reading the question

When the trigger words aren't obvious, and when one question needs several patterns.

START
TRICKY CASES

Still unsure? Ask what the problem is about

Making objects?	Creational. One object → Singleton. Many types → Factory.
Wrapping or connecting objects?	Structural. Use the wrapper test →
Who does what, and in what order?	Behavioural. Fixed steps → Template. Passed along → Chain. Changes with state → State.

The wrapper test

Four patterns wrap something. Ask what the wrapper does.

Adapter	**changes** the interface (a plug adapter)
Facade	**simplifies** many classes into one (a waiter)
Proxy	same interface, **controls** access (a security guard)
Decorator	same interface, **adds** features, stackable (toppings)

Pairs people mix up

one line each, good for "compare" questions

State vs Chain	State asks "how should I behave right now?" The object keeps one current state and hands work to it. Chain asks "who will handle this?" The request moves along a line of handlers.
Template vs Strategy	Template Method fixes the steps in a parent class and lets subclasses fill some of them (inheritance). Strategy swaps the whole algorithm for another object at runtime (composition).
Factory vs Singleton	Factory decides **which type** of object to create. Singleton decides **how many** (just one). They are often used together.
Factory vs Facade	Factory creates objects. Facade uses objects that already exist and gives them one simple front.
Proxy vs Decorator	Both keep the same interface and wrap one object. Proxy controls access and is usually one layer. Decorator adds features and is stacked many times.
Adapter vs Facade	Adapter converts one interface into another that the client expects. Facade puts one new, simpler interface over many classes.

One scenario, several patterns

how 2025 asked every question

`2025 Q1` kiosk	kiosk states → **State 09** · optional add-ons → **Decorator 06** · one tax calculator → **Singleton 01**
`2025 Q3` library	checks in order → **Chain 08** · types of book → **Factory 02** · rare books → **Proxy 05** · optional services → **Decorator 06**
`2025 Q4` shop	same payment steps → **Template 07** · order checks → **Chain 08** · order IDs → **Singleton 01** · payment objects → **Factory 02** (fully worked in tab 13)
`2022 Q1` travel	combine air, hotel and taxi → **Facade 03** · each third-party API → **Adapter 04**
`Quizzes`	parking: State + Adapter · database: Factory + Singleton · scanner: Adapter + Facade · course: Template + Decorator · support desk: Chain + Template

How the in-class quizzes were structured

1 why this pattern (2–4m) → **2** draw it: state and/or class diagram (4–8m) → **3** Java: interface, abstract or context class (3–6m) → **4** a second pattern (3–5m): parking → Adapter, scanner → Facade, course → Decorator, support desk → Template. In 2024 and 2025 the paper also named the classes and methods to write; when it does, use those names exactly.

00 Past-question lookup

Every case from the four past papers (2022–2025) and the seven in-class quizzes, with the pattern it used and where the answer is in this note.

START
LOOKUP

SOURCE	CASE	PATTERN(S) USED	TAB
2022 Q1	Travel booking: air, hotel, taxi from third-party services	Facade + Adapter	03, 04
2022 Q2	Supermarket pay for executives, daily workers, temporary staff	Template Method	07
2022 Q3	Email spam filter with sub-filters; HA and DR	Chain of Responsibility	08, 12
2022 Q4	Cake options · product framework · payroll by one user · LMS pages · LMS modules	Decorator · Factory · Singleton · Proxy · Factory	06 02 01 05
2023 Q1	Salary components; configurable parameters; no one else logged in during payroll	Template Method · config · Singleton + lock	07, 01
2023 Q2	Supermarket chain: purchasing, sales, warehousing	Microservices	11
2023 Q3	Coffee flavours · event websites from other vendors · logging · HA/DR	Decorator · Adapter · Singleton	06 04 01
2023 Q4	Fantasy cricket · bank accounts · exception priorities · LMS pages · authorization	Multiton · Factory · Chain · Proxy · Proxy	01 02 08 05
2024 Q1	Coffee add-ons, Coffee interface, BasicCoffee, profit per coffee, logger	Decorator + Singleton	06, 01
2024 Q2	Library management system	Microservices	11
2024 Q3	Document workflow: Draft, Moderation, Published, Archived	State	09
2024 Q4	E-commerce checkout steps; exceptions; abstract classes; authorization	Template · Chain · Proxy	07 08 05
2025 Q1	Self-checkout kiosk: states, add-ons, one tax calculator	State · Decorator · Singleton	09 06 01
2025 Q2	Media streaming platform	Microservices	11
2025 Q3	Library borrowing: checks, book types, rare books, optional services	Chain · Factory · Proxy · Decorator	08 02 05 06
2025 Q4	E-commerce orders: payment workflow, checks, Order IDs, payment objects	Template · Chain · Singleton · Factory	13
Quiz 1	Smart parking session states; legacy payment machine	State + Adapter	09, 04
Quiz 2	MySQL / PostgreSQL / MongoDB objects; one DB connection	Factory + Singleton	02, 01
Quiz 3	New barcode scanner; one borrowBook() over four services	Adapter + Facade	04, 03
Quiz 4	Smart printing: many PrinterManager objects	Singleton	01
Quiz 5	User profile: premium badge, 2FA, theme	Decorator	06
Quiz 6	Online course completion; optional AI assistant / premium / mentor	Template + Decorator	07, 06
Quiz 7	Technical support levels; enforced log → solve → log procedure	Chain + Template	08

B Design pattern basics

Lecture 1 theory, for "explain" questions and for opening an answer.

What is a design pattern?

> A design pattern is a proven, reusable solution to a problem that keeps coming up in software design. It is not finished code. It is a template for how to arrange classes and objects, which you adapt to your own situation.

Every pattern has **four elements**: a **name** (so developers can talk about it), the **problem** and its context (when to use it), the **solution** (the classes, their roles and how they relate), and the **consequences** (the benefits and the costs).

Where they came from

- The idea started in building architecture. **Christopher Alexander** wrote about reusable patterns for towns and buildings.
- In **1995**, four authors (Gamma, Helm, Johnson and Vlissides, the **"Gang of Four"** or GoF) published *Design Patterns: Elements of Reusable Object-Oriented Software*.
- The book describes **23 patterns** in three groups. These are the "GoF patterns".

Why use them

- They give the team a **shared vocabulary**: "use a Factory here" says a lot in three words.
- They make code **flexible and reusable**, with **loose coupling**.
- They are **easier to maintain and extend**, because changes stay in one place.
- They **avoid repeating design mistakes** that others have already solved.

The three groups

bold = taught in this module

GROUP	WHAT IT DEALS WITH	PATTERNS
Creational	How objects are created, and who creates them	**Singleton**, **Factory Method**, Abstract Factory, Builder, Prototype
Structural	How classes and objects are put together into bigger structures	**Adapter**, **Facade**, **Proxy**, **Decorator**, Composite, Bridge, Flyweight
Behavioural	How objects talk to each other and share responsibilities	**Template Method**, **Chain of Responsibility**, **State**, Strategy, Observer, Command, Iterator

What patterns are not

- Not ready-made code you can paste anywhere.
- Not guaranteed to be correct, and not rules you must always follow.
- Not something to force on a simple problem. Using one where it isn't needed is **over-engineering**.

Anti-patterns

An **anti-pattern** is a common solution that looks good now but causes problems later. It's a poor answer to a recurring problem.

Why they happen: time pressure, poor planning, overconfidence, using new technology just to impress, no design thinking, ignoring maintenance.

Examples: a God class that does everything, copy-pasted code, long if/else chains on a type field.

Patterns, frameworks and libraries

the "Frameworks" in the module name

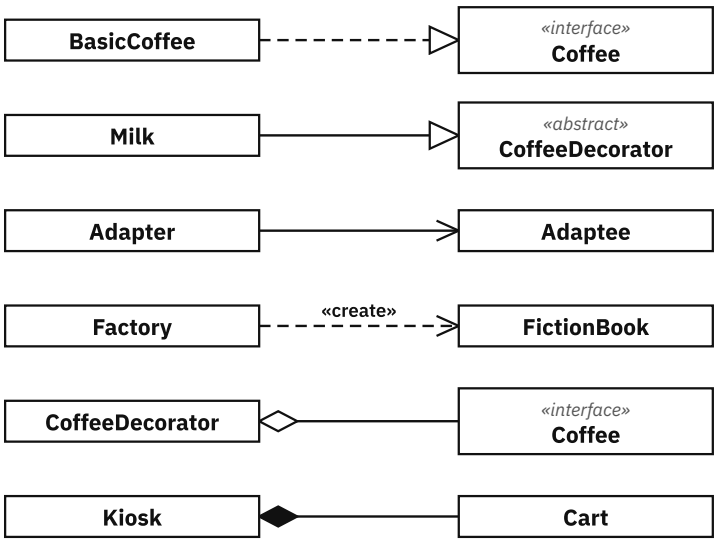
Pattern	An idea for solving a design problem. No code of its own. You implement it yourself.
Library	Reusable code that **your code calls** when it needs something (e.g. a JSON parser).
Framework	Reusable infrastructure that **calls your code** ("don't call us, we'll call you", called **Inversion of Control**). Frameworks are built from patterns: Spring uses Dependency Injection and Proxy, JUnit uses Template Method, Java I/O uses Decorator and Adapter.

B Drawing UML in the exam

Diagrams carry 4 to 10 marks. The arrows are where people lose marks, so check them here.

The six arrows

the shape always sits at the end shown here



Realisation: a class implements an interface. Dashed line, hollow triangle at the interface.

Generalisation: a class extends a parent class. Solid line, hollow triangle at the parent.

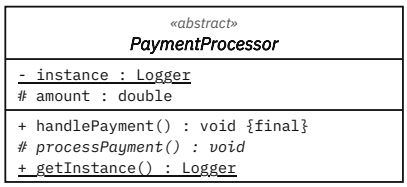
Association: a class holds a field of the other type. Solid line, open arrow at the one being used.

Dependency: a class uses or creates the other, no field. Dashed line, open arrow. Label it «create» for factories.

Aggregation: "has-a", the part can exist on its own. Hollow diamond at the owner.

Composition: "owns", the part dies with the owner. Filled diamond at the owner.

Reading a class box



← stereotype and name (italic = abstract)

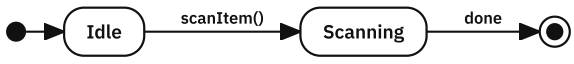
← fields: visibility name : Type underlined = static

← methods: + public, # protected italic = abstract method

Symbols inside the box

`+`	public
`–`	private
`#`	protected
underlined	static (belongs to the class)
italic	abstract class or method
«interface»	written above the name
{final}	can't be overridden
`name : Type`	field, type after the colon
`op(p) : Type`	method and return type

State diagram symbols



Filled dot = start. Rounded box = state. Arrow with the event that causes it. Bullseye = end (only if the object really finishes).

Drawing a class diagram in 5 steps

1. Draw the **interface or abstract class** at the top, in the middle.
2. Draw the **concrete classes** under it. Connect with hollow-triangle arrows pointing up.
3. Add the **"has-a" links** (diamonds, or arrows for fields).
4. Put the **client** on the left with an arrow to what it uses.
5. Fill in only the **important** fields and methods. Use their names from the question.

01 Singleton

Only one object of the class can exist, and everyone uses that same object.

CREATIONAL
UNDERSTAND

Spot it: the question says...

only one instance · a single global tax calculator · one logger, all logs in the same file · every order gets a unique ID · only one user can run the payroll · only one DB connection active · a new object is created every time → duplicates, wrong order · maximum of N → Multiton

The idea

Some things should exist only once in a system: the print queue for the one printer, the logger, the configuration, the tax calculator. If every part of the program creates its own copy, each copy has its own data, and they disagree.

Singleton stops that. The class creates its one object itself and hands the same object to anyone who asks. Think of an airport control tower: every plane talks to the same tower.

What goes wrong without it

Quiz 4

- **Inconsistent data**: each object has its own queue or status, so departments see different printer statuses.
- **Resource conflicts**: several objects send jobs at once, so jobs print in the wrong order, get duplicated, or the printer freezes.
- **Wasted memory** and no central control.

The three ingredients

name all three in your answer

1
Private constructor
No other class can write `new PrinterManager()`.

2
Private static field
Holds the one instance. Static means it belongs to the class, so there is only one copy.

3
Public static getInstance()
Creates the object the first time, then always returns the same one.

Also make the class **final** so no subclass can get around the private constructor.

How many objects do you need?

exactly 1	**Singleton** (DB manager, logger)	N reusable	**Object pool** (10 DB connections)
at most N	**Multiton**, "extended Singleton" (3 printers, 5 batsmen)	unlimited	a normal class (customers)

Asked before

WHERE	WHAT THEY ASKED	ANSWER
`Quiz 4` 5m	Many PrinterManager objects: two problems, and the fix	**above + W1**
`Quiz 2` Q4	Make sure only one DB connection is active at a time	**answers**
`2022` Q4.3 · `2023` Q1.6	Payroll runs for one user only, block everyone else	**answers**
`2023` Q3.4 · `2024` Q1.5	A logger that writes everything to one file	**answers**
`2023` Q4.1 5m	Fantasy cricket: max 5 batsmen, 5 bowlers, 1 keeper, 1 captain, using an "extended Singleton"	**answers**
`2025` Q1 · Q4	One global tax calculator / one OrderID generator, and how it fits the flow	**answers**

01 Singleton: write

Copy these blocks in order. Replace PrinterManager with the class from your question.

CREATIONAL
WRITE

W1 Name it and say why

2–5 marks

> I would use the **Singleton** pattern, which is a creational pattern. It makes sure a class has **only one instance** and provides **one global point of access** to it.

Reasons (pick two or three)

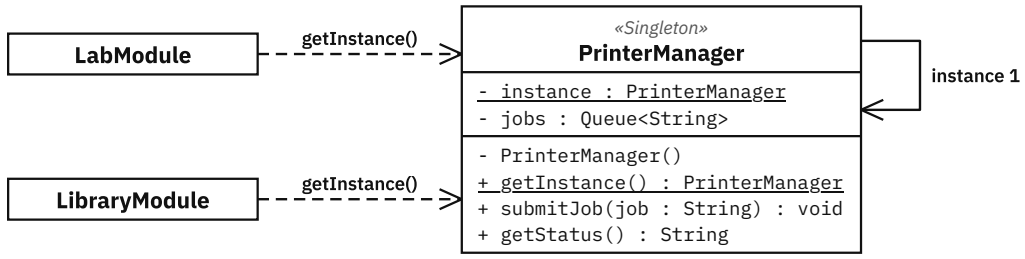
- Everyone works with the **same object**, so there is one queue / one status, and the data stays consistent.
- No duplicate objects fighting over the same resource, so no duplicated or out-of-order jobs.
- **Saves memory**, because the object is created once and reused.

Map it to the scenario

- **PrinterManager** is the Singleton class.
- The **lab, library, engineering and staff room modules** are the clients.
- Each client calls `PrinterManager.getInstance()` instead of `new`, so all print jobs go into one queue.

W2 Class diagram

underline the static field and method



W3 · JAVA (LAZY AND THREAD-SAFE)

```java
public final class PrinterManager {
    // the one shared object (volatile for threads)
    private static volatile PrinterManager instance;
    private final Queue<String> jobs = new LinkedList<>();

    // private: nobody outside can call "new"
    private PrinterManager() { }

    // the global access point
    public static PrinterManager getInstance() {
        if (instance == null) {              // check 1
            synchronized (PrinterManager.class) {
                if (instance == null) {      // check 2
                    instance = new PrinterManager();
                }
            }
        }
        return instance;
    }

    public synchronized void submitJob(String job) {
        jobs.add(job);
    }
}

// client: every department gets the same object
PrinterManager p1 = PrinterManager.getInstance();
PrinterManager p2 = PrinterManager.getInstance();
System.out.println(p1 == p2);    // true
```

W4 Thread-safe versions

Eager	Created when the class loads. Safe and simple, but created even if never used.
Lazy	**Not** thread-safe. Two threads can both see null and make two objects.
synchronized getInstance()	Safe, but locks on every call, so slower.
Double-checked	Lazy, safe and fast. This is the code on the left.

Eager in one line: `private static final X INSTANCE = new X();`

W5 Challenges → fix

- **Two threads create two objects** → double-checked locking or eager creation.
- **Hard to unit test** (can't swap in a mock) → let it implement an interface and inject it.
- **Single point of failure / bottleneck** → Multiton or a pool, plus a standby replica (HA).
- **Overuse as a global variable** → only use it for things that are truly single.

01 Singleton: past questions answered

Each answer is written at the length the marks need.

One DB connection at a time

Quiz 2 Q4

Make a **DatabaseConnectionManager** Singleton. Its constructor is private and it keeps one static `connection`. `getConnection()` opens the connection only if it is null and otherwise returns the one that exists. The method is synchronized. The Factory picks the database type once, and everyone reuses that single connection.

Logger, all logs in one file

2024 Q1.5 5m

Use a **Singleton Logger**. The one instance owns the only `FileWriter`, and every class calls `Logger.getInstance().log(msg)`. So there is one file handle (no corrupted or duplicate lines), one format, one order, and less memory. `log()` is synchronized so lines from different threads don't mix.

2023 Q3.4 "implement logging": the same answer works. You can add that a **logging Proxy** could add logging around existing services without changing them.

Payroll runs for one user only

2022 Q4.3 · 2023 Q1.6, 4–5m

A Singleton alone does not give exclusive access. It only guarantees there is **one** `PayrollEngine` object, and therefore one lock that every caller shares. The exclusion comes from that lock. Two behaviours are possible, and the question decides which: **reject immediately** (`tryLock()` returns false at once, "payroll is running, try later") or **wait** (`lock()` blocks until the current run finishes). Ownership is released in `finally`, so an exception during the calculation can't leave the system locked forever.

```
public final class PayrollEngine {
  private static final PayrollEngine INSTANCE =
      new PayrollEngine();
  private final ReentrantLock lock =
      new ReentrantLock();   // java.util.concur
  private PayrollEngine() { }
  public static PayrollEngine getInstance() {
    return INSTANCE; }

  // option 1: refuse at once if busy
  public boolean tryRunPayroll(String user) {
    if (!lock.tryLock()) return false;
    try { calculateAll(user); return true; }
    finally { lock.unlock(); }   // always relea
  }
```

```
  // option 2: wait until the current run ends
  public void runPayrollWhenFree(String user) {
    lock.lock();
    try { calculateAll(user); }
    finally { lock.unlock(); }
  }
  public boolean isRunning() {
    return lock.isLocked(); }
  private void calculateAll(String user) {
    /* calculate every employee's salary */ }
}

// every OTHER entry point must ask the same obj
if (PayrollEngine.getInstance().isRunning())
  throw new IllegalStateException(
      "Payroll in progress, login refused");
```

Coordinated access checks: "no other user logged in, no other process running" (2023 Q1.6) is only true if the **login service, data-entry screens and batch jobs all check the same lock** before they start (last snippet). Anything that skips the check is not blocked. The lock lives inside one running program (JVM). If the system runs on **several servers**, each has its own Singleton, so use a shared lock instead, such as a database row lock (`SELECT … FOR UPDATE` on a `payroll_run` row) checked by every server.

01 Singleton: more answers

How a Singleton plugs into a bigger design, and how to make settings configurable.

One global tax calculator

2025 Q1.5, 4m

Each checkout counter adds up the items' `getCost()` (the Decorator gives the final item prices). In the Payment state the kiosk calls `TaxCalculator.getInstance().calculateTax(subtotal)`. Every counter gets the same instance, so they all use one rate and one set of rules. Change the rate once and it changes everywhere.

OrderID generator

2025 Q4, 5m

Use an eager Singleton with an `AtomicLong` counter, so two threads asking at the same moment never get the same ID. The first step of the template method `handlePayment()` is `OrderIdGenerator.getInstance().nextId()`, then validate → process → send receipt. All payment types share the one generator.

```
public final class OrderIdGenerator {
  private static final OrderIdGenerator
      INSTANCE = new OrderIdGenerator();
  private final AtomicLong counter =
      new AtomicLong(1000);
  private OrderIdGenerator() { }
  public static OrderIdGenerator
      getInstance() { return INSTANCE; }
  public String nextId() {
    return "ORD-"
        + counter.incrementAndGet();
  }
}
```

Scope of "unique": only **within one running program**. After a restart the counter starts at 1000 again, and two servers each have their own Singleton and counter, so both would repeat IDs. For real uniqueness, start the counter from the last saved ID, or take IDs from a **database sequence** (or a central ID service, or a server prefix such as `ORD-S2-1001`).

"Parameters change over time. Improve configurability."

2023 Q1.5, 4m

Take every rate and limit out of the code (EPF 8%, employer 12%, ETF 3%, tax threshold 100,000, tax 6%) and keep them in **external configuration**: a database table or a properties / YAML file.

A **Singleton ConfigManager** loads them once and every calculation reads from it. An admin screen updates the values. Changes are versioned and audited, with an "effective from" date. No recompiling or redeploying is needed.

```
# payroll.properties
epf.employee=0.08
epf.employer=0.12
etf=0.03
tax.threshold=100000
tax.rate=0.06

// in the calculation
double rate = ConfigManager.getInstance()
                  .get("epf.employee");
double epf = gross * rate;
```

01 Singleton: Multiton, OOP and SOLID

The "extended Singleton" answer, what each part of the code does in OOP terms, and mistakes to avoid.

CREATIONAL
ANSWERS · THEORY

Fantasy cricket with an "extended Singleton"

2023 Q4.1, 5m

Use a **Multiton**: a Singleton that allows a fixed maximum number of objects per key. Player objects can only be created through `addPlayer(role, name)`, which counts how many of each role exist and refuses once the limit is reached: 5 batsmen, 5 bowlers, 1 wicketkeeper.

The **captain** is a plain Singleton, because there must be exactly one:

```
public final class Captain {
  private static Captain instance;
  private Player player;
  private Captain() { }
  public static synchronized Captain
      getInstance() {
    if (instance == null)
      instance = new Captain();
    return instance; }
  public void appoint(Player p) {
    player = p; }  // replaces, never a
}
```

```
public final class Player {
  private static final Map<String,Integer> LIMIT =
    Map.of("BATSMAN",5, "BOWLER",5, "KEEPER",1);
  private static final Map<String,List<Player>> team
    = new HashMap<>();
  private final String role, name;
  private Player(String role, String name) {
    this.role = role; this.name = name; }

  public static synchronized Player addPlayer(
      String role, String name) {
    if (!LIMIT.containsKey(role))
      throw new IllegalArgumentException(role);
    team.putIfAbsent(role, new ArrayList<>());
    if (team.get(role).size() >= LIMIT.get(role))
      throw new IllegalStateException("Max reached");
    Player p = new Player(role, name);
    team.get(role).add(p);
    return p;
  }
}
```

OOP and SOLID in the Singleton code

name the concept → class/member → effect

CONCEPT	WHERE (PRINTERMANAGER)	EFFECT
Encapsulation	`private PrinterManager()`, `private … jobs`	Only the class can create itself; the queue is reached only through `submitJob()`, so no one can corrupt it.
Class (static) members	`private static volatile instance`, `public static getInstance()`	Static belongs to the class, so there is one copy for the whole program, reachable without an object.
Inheritance blocked	`public final class`	No subclass can add a public constructor and create a second object.
Thread safety	`volatile`, `synchronized` block, `synchronized submitJob()`	Only one object is created even under concurrent calls; jobs are added one at a time.
SRP (tension)	the class manages print jobs *and* its own creation	Two reasons to change. Accepted for small shared resources.
DIP (risk)	clients call `PrinterManager.getInstance()` directly	A hidden, concrete dependency, hard to mock. Fix: let it implement `PrintQueue` and pass the instance into clients through their constructors.
OCP	`final` class	It can't be extended by subclassing; change it by editing, or put it behind an interface.

Often combined with

Factory	The factory picks the DB type; the connection manager is a Singleton.
Facade	Only one facade is needed, so make it a Singleton.
State	State objects with no data can be shared Singletons.

Mistakes that lose marks

- Leaving the constructor public.
- Making `getInstance()` non-static.
- Not underlining static members in the class diagram.
- Saying the basic lazy version is thread-safe.
- Using a Singleton when the question says "maximum N" (that's a Multiton).
- Calling it a structural pattern. It's creational.

02 Factory Method

One place decides which class to create. The rest of the code just asks for "a database" or "a book".

CREATIONAL
UNDERSTAND

Spot it: the question says...

- create different types of X depending on the user's request
- different types of accounts / books / payments
- supports MySQL, PostgreSQL and MongoDB
- too many if-else blocks to create objects
- without exposing the concrete class
- scalable, extensible product-based framework
- modules created depending on the user's needs

The idea

When the program needs one of several related types, the code that uses the object shouldn't have to know which class to `new`. A factory takes the request ("MYSQL", "FICTION", "CARD") and hands back the right object, typed as the common interface.

Like ordering at a restaurant: you ask for "a pizza", and the kitchen decides how to make it. You never go into the kitchen.

The problem it removes

```
// BEFORE: copied all over the client
if (type.equals("MYSQL"))
    db = new MySQLDatabase();
else if (type.equals("MONGO"))
    db = new MongoDatabase();
// new database = edit every copy

// AFTER: one line, interface only
Database db =
    DatabaseFactory.getDatabase(type);
```

The parts

name them in your answer

PART	JOB	EXAMPLE
Product «interface»	The common type every product shares	`Database`, `Book`, `Account`
Concrete products	The real classes	`MySQLDatabase`, `FictionBook`, `SavingAccount`
Factory / Creator	Decides which concrete class to create and returns it as the Product	`DatabaseFactory.getDatabase(type)`
Client	Asks the factory, then uses the product through the interface only	`ProductService`

Don't mix it up

- **vs Singleton**: Factory decides which type; Singleton decides how many (one). Often used together: the factory or the connection it creates is a Singleton.
- **vs Facade**: Factory creates objects. Facade coordinates objects that already exist.
- **vs Abstract Factory**: Abstract Factory creates whole families of related objects (e.g. a Windows button + Windows menu). Not taught in detail.

Why is `new` in the client bad?

It hard-codes the concrete class. Every new type forces a change in the client, which breaks **Open/Closed**, and the client depends on details instead of an abstraction, which breaks **Dependency Inversion**. The factory also separates creating from using (**Single Responsibility**).

Asked before: `Quiz 2` database types · `2025` Q3 book types (Book interface + BookFactory, 7m) · `2025` Q4 payment objects · `2023` Q4.2 bank accounts · `2022` Q4.2, 4.5 product framework / LMS modules. **All answered on the answers page.**

02 Factory Method: write

Copy in order. Swap Database and its types for the product in your question.

CREATIONAL
WRITE

W1 Name it and say why

2–5 marks

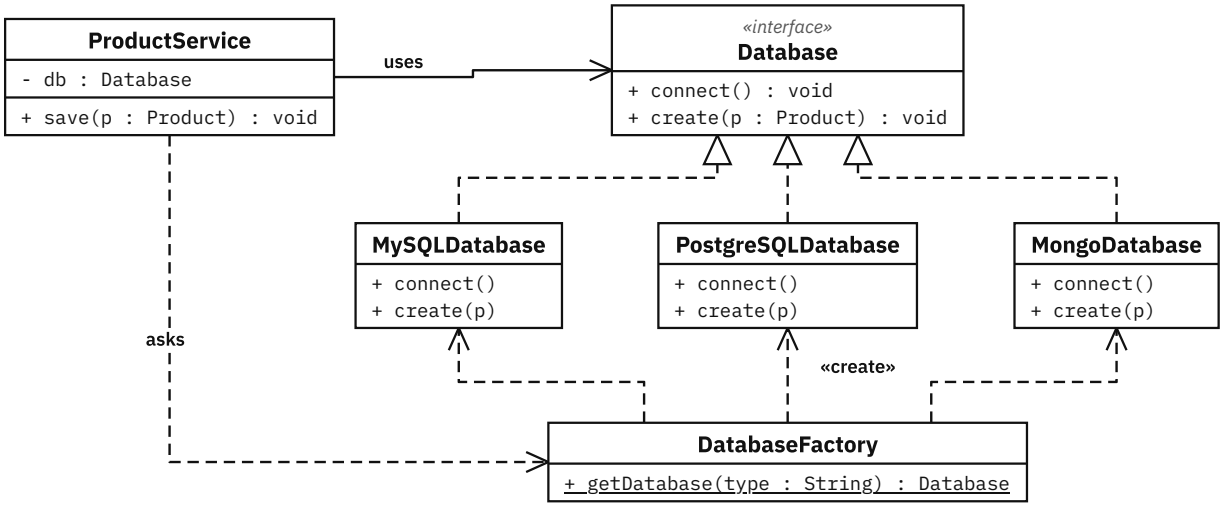
> I would use the **Factory Method** pattern, which is a creational pattern. It defines an interface for creating an object but lets the factory (or a subclass) decide **which class to instantiate**, so the client never uses `new` on a concrete class.

Reasons: (1) **loose coupling**, because the client depends only on the `Database` interface, not on MySQL or MongoDB classes; (2) the **creation logic is in one place** instead of if-else copied across the code; (3) it is **easy to extend (Open/Closed)**: a new type is one new class and one new case, and client code doesn't change.

Mapping: Product = **Database** «interface» · concrete products = **MySQLDatabase, PostgreSQLDatabase, MongoDatabase** · factory = **DatabaseFactory.getDatabase(type)** · client = **ProductService**.

W2 Class diagram

dashed «create» arrows from the factory; dashed triangles to the interface



W3 · JAVA

```java
// 1. PRODUCT
public interface Database {
    void connect();
    void create(Product p);
}
// 2. CONCRETE PRODUCTS
public class MySQLDatabase
        implements Database {
    public void connect() {
        /* open MySQL connection */ }
    public void create(Product p) {
        /* INSERT into MySQL */ }
}
// PostgreSQLDatabase, MongoDatabase:
// same shape
```

```java
// 3. FACTORY: the only place with "new"
public class DatabaseFactory {
  public static Database getDatabase(
          String type) {
    switch (type.toUpperCase()) {
      case "MYSQL": return new MySQLDatabase();
      case "POSTGRES":
          return new PostgreSQLDatabase();
      case "MONGO": return new MongoDatabase();
      default: throw new
        IllegalArgumentException(type);
    }
  }
}
// 4. CLIENT: knows only the interface
Database db =
    DatabaseFactory.getDatabase("MYSQL");
db.create(product);
```

02 Factory Method: past questions answered

Short answers sized to the marks.

Database objects

Quiz 2 Q1–Q3

Role of the factory	It takes object creation out of the client. The client asks `DatabaseFactory.getDatabase("MYSQL")` and receives a `Database`. The factory decides and creates MySQL, PostgreSQL or Mongo.
Two advantages	(1) **Loose coupling**: switching from MySQL to PostgreSQL changes only the parameter or config, not the business code. (2) **Easy to extend**: adding Oracle is one new class and one new case, and the creation logic stays in one place.
Why the developer doesn't need to know the DB	Because of **abstraction and polymorphism**. The variable's type is the `Database` interface, and every database implements `connect()` and `create()`. The factory returns the right object at runtime, and `db.create(product)` runs the right version automatically.

Library book types: Book interface + BookFactory

2025 Q3.2–3.3, 7m

```
public interface Book {
    String getTitle();
    String getCategory();
}
public class FictionBook implements Book {
    private String title;
    public FictionBook(String t) {
        title = t; }
    public String getTitle() {
        return title; }
    public String getCategory() {
        return "Fiction"; }
}
// ResearchBook, Magazine: same shape
```

```
public class BookFactory {
  public static Book createBook(
          String type, String title) {
    switch (type) {
      case "FICTION":
          return new FictionBook(title);
      case "RESEARCH":
          return new ResearchBook(title);
      case "MAGAZINE":
          return new Magazine(title);
      default: throw new
          IllegalArgumentException(type);
    }
  }
}
// client
Book b = BookFactory.createBook(
        "RESEARCH", "AI Ethics");
```

Bank accounts

2023 Q4.2, 5m

Factory Method. An `Account` interface (or abstract class) with `deposit()`, `withdraw()` and `calculateInterest()`. `CurrentAccount`, `SavingAccount` and `ChildrenAccount` implement it with their own rules (overdraft, interest, withdrawal limit). `AccountFactory.createAccount(type)` returns the right one. The bank code only uses `Account`, so a new FixedDeposit account can be added without touching it.

Payment objects

2025 Q4.1(d)

Factory Method. `PaymentFactory.create(method)` returns `CreditCardProcessor`, `PayPalProcessor` or `CODProcessor`, all typed as `PaymentProcessor`. The customer picks the method at runtime, so the client needs no `new` and no if-else. Each processor then follows the Template Method `handlePayment()` (tab 07).

02 Factory Method: more answers

Short 5-mark answers, the names to swap in, and the challenges.

LMS modules / product framework

2022 Q4.2, Q4.5

Factory Method. A `Module` interface, implemented by Quiz, Forum, Assignment and VideoLecture. `ModuleFactory.create(userNeed)` builds what the user needs. New modules (new products) are new classes registered with the factory, so the framework grows without changing existing code. That is what "scalable, extensible product-based framework" is asking for.

"It avoids code duplication." Agree?

Yes. The creation logic (the if-else and the `new` calls) is written once in the factory instead of in every client. Shared behaviour can sit in an abstract creator. The cost is extra classes, so keep the factory parameterised rather than writing near-identical creators.

Mistakes: client still calls `new MySQLDatabase()`; factory returns the concrete type instead of the interface; missing «create» arrows.

W4 Rename for your question

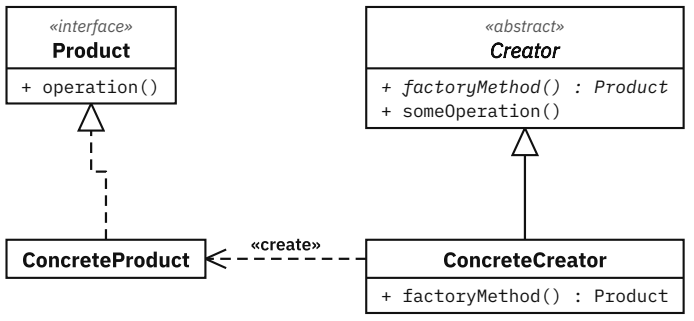
PRODUCT	CONCRETE PRODUCTS	FACTORY
`Book`	Fiction, Research, Magazine	BookFactory
`Account`	Current, Saving, Children	AccountFactory
`PaymentProcessor`	CreditCard, PayPal, COD	PaymentFactory
`Module`	Quiz, Forum, Assignment	ModuleFactory

W5 Challenges → fix

- **More classes** → keep one parameterised factory, not a creator per type.
- **The switch is edited for each new type** → register types in a `Map` or config.
- **Unknown type** → validate input, throw a clear exception.
- **Need one shared factory or connection** → make it a Singleton.

The strict GoF version

subclasses decide



Two forms

Simple factory (what the lectures and past answers use): one class with a static `create(type)` method and a switch.

GoF Factory Method: an abstract Creator declares `factoryMethod()`, and each ConcreteCreator subclass returns its own product.

Either is accepted. Use the simple one unless the question says "let subclasses decide".

03 Facade

One simple front door to a group of complicated classes. The client talks to the front door only.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

one simple interface for borrowing books by internally calling inventory, payment, notification and login

combine air tickets, hotels and taxis | single entry point | hide the complexity of the subsystems

complex subsystem calls scattered everywhere

The idea

Some jobs need many classes called in the right order: log in, check stock, charge a fee, send a message. If every client does that itself, each one depends on all those classes, and any change breaks them all.

A facade puts one method in front, like `borrowBook()`, which makes the calls in the right order. Like a waiter: you order from them and never deal with the chef, the cashier and the kitchen staff yourself.

Without vs with

	WITHOUT FACADE	WITH FACADE
Client talks to	Login, Inventory, Payment, Notification...	one class
Coupling	high, many dependencies	low
A service changes	every client breaks	only the facade changes
Call order	repeated in each client	written once

The parts

PART	JOB	LIBRARY EXAMPLE
Facade	Knows which subsystem does what, calls them in order. Only delegates.	`LibraryFacade.borrowBook()`
Subsystem classes	Do the real work. They don't know the facade exists.	`LoginService`, `InventoryService`, `PaymentService`, `NotificationService`
Client	Calls only the facade.	librarian screen / mobile app

Rules from the slides

- A facade **doesn't add new functionality**. It only gives a simpler way in.
- Clients **can still use the subsystems directly** if they need something advanced.
- A big system **can have more than one facade**, one per area (BookingFacade, PaymentFacade).

Don't mix it up

- **vs Adapter**: Adapter converts one interface into another so the client can use it. Facade makes a new, simpler interface over many classes. Adapter = translator, Facade = simplifier.
- **vs Factory**: Factory creates objects. Facade coordinates objects that already exist.

Asked before: `2022` Q1 travel booking (diagram, which pattern, challenges and how to overcome them, 15m) · `Quiz 3` Q5 one interface for borrowing books. **Answered on the answers page.**

03 Facade: write

Copy in order. Swap the services for the ones in your question.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

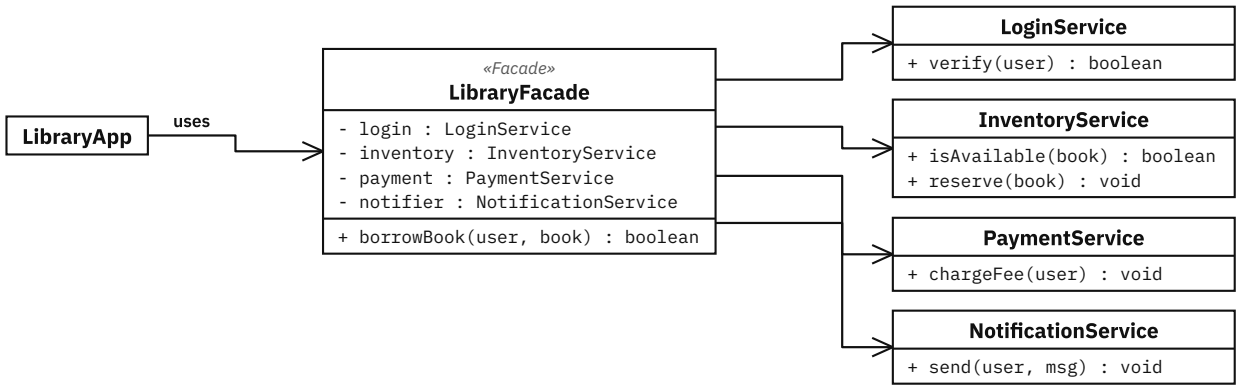
> I would use the **Facade** pattern, which is a structural pattern. It provides **one unified, simpler interface** to a set of interfaces in a subsystem, so the subsystem is easier to use.

Reasons: (1) the client depends on **one class instead of four services**, so coupling is low; (2) the **call order is written once** inside the facade instead of being repeated in every screen; (3) if a service changes, **only the facade changes** and the clients don't; (4) it is easier to use and test, because you can mock the facade.

Mapping: Facade = **LibraryFacade** with `borrowBook()` · subsystems = **LoginService, InventoryService, PaymentService, NotificationService** · client = **LibraryApp**.

W2 Class diagram

arrows go from the facade to the subsystems, never back



W3 · JAVA

```java
// SUBSYSTEMS: do the real work
class LoginService {
  boolean verify(String user) { ... }
}
class InventoryService {
  boolean isAvailable(String book) { ... }
  void reserve(String book) { ... }
}
class PaymentService {
  void chargeFee(String user) { ... }
}
class NotificationService {
  void send(String user, String msg) { ... }
}

// CLIENT: one call
LibraryFacade library = new LibraryFacade();
library.borrowBook("U001", "B123");
```

```java
// FACADE: one simple method, calls in order
public class LibraryFacade {
  private LoginService login =
      new LoginService();
  private InventoryService inventory =
      new InventoryService();
  private PaymentService payment =
      new PaymentService();
  private NotificationService notifier =
      new NotificationService();

  public boolean borrowBook(String user,
                            String book) {
    if (!login.verify(user)) return false;
    if (!inventory.isAvailable(book))
        return false;
    inventory.reserve(book);
    payment.chargeFee(user);
    notifier.send(user, "Borrowed " + book);
    return true;
  }
}
```

03 Facade: answers and challenges

The travel booking question in full, and the challenges with their fixes.

STRUCTURAL
ANSWERS

Travel booking: air ticket, hotel and taxi

2022 Q1.1–1.2, 15m

1.1 Diagram: draw the scenario diagram from tab 00 (traveller → website → TravelBookingFacade → air / hotel / taxi services → third-party APIs through adapters).

1.2 Pattern for the sub-system that combines them: Facade. `TravelBookingFacade.bookTrip(request)` calls the AirTicketService, HotelService and TaxiService and returns one itinerary. The traveller does one action, the complexity of three services is hidden, and the website depends on one class only.

1.2.1 CHALLENGE	1.2.2 MECHANISM TO OVERCOME IT
The facade grows into a **God class** as more services are added.	Split it into smaller facades per area (FlightFacade, StayFacade). Keep it delegating only, no business logic (SRP).
One service fails (taxi API down) and the **whole booking fails**. The facade is a **single point of failure**.	Timeouts, retries and a fallback provider for each service. Run the facade as several stateless copies behind a load balancer (HA).
Partial booking: the flight is booked but the hotel fails.	Compensating actions: cancel the flight if the hotel fails (the saga idea), or reserve everything first and confirm all at the end.

1.3 (connecting to the different third-party web services) is Adapter: see tab 04, answers.

One interface for borrowing

Quiz 3 Q5

Use a **Facade**. `LibraryFacade.borrowBook(userId, bookId)` calls, in order: `LoginService.verify()` → `InventoryService.isAvailable()` and `reserve()` → `PaymentService.chargeFee()` → `NotificationService.send()`. The librarian screen calls only the facade, so it doesn't depend on four services, and a change in any service is handled inside the facade. (Code: W3.)

Slide questions

Can you still access the subsystems directly? Yes. The facade is a convenience, not a lock.

Must there be only one facade? No. A large system can have several, one per area, which also avoids a God class.

Does the facade add functionality? No. It only wraps existing features behind a simpler entry point.

W5 Challenges → fix

use for any "challenges" part

CHALLENGE	HOW TO HANDLE IT
Becomes a **God class** (too big)	Several smaller facades; the facade only delegates.
Single point of failure / bottleneck	Keep it stateless and run several copies behind a load balancer. If one is enough, make it a Singleton.
Can **hide useful features** of the subsystems	Still allow direct access for advanced clients.
Subsystems have **incompatible interfaces** (third-party)	Put Adapters under the facade (tab 04).
Extra layer, not worth it for tiny systems	Only use it when several clients repeat the same call sequence.

Mistakes that lose marks: putting business logic in the facade (it only delegates) · drawing arrows from the subsystems back to the facade (they don't know it) · calling it creational · confusing it with Adapter (converts an interface) or Proxy (same interface, controls access).

04 Adapter

A translator class that lets your code use a class whose methods don't match what your code expects.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

legacy system · supplied by an external vendor, can't modify · the new scanner provides readBarcode() instead of scanBook() · hundreds of calls to the old method · third-party web services with different method signatures · websites developed by different vendors and technologies · incompatible interfaces

The idea

Your system calls `scanBook()` in hundreds of places. The new scanner only has `readBarcode()`. You can't change the scanner (vendor code) and you don't want to change hundreds of calls.

So you write one small class that **looks like** the old scanner (it has `scanBook()`) and inside it calls `readBarcode()`. Like a travel plug adapter: the socket and the plug stay as they are, and the adapter sits between them.

Why not just change the code?

Quiz 3 Q1

- **Hundreds of call sites**: a huge change with a high risk of new bugs, full re-testing and downtime.
- Tested, working code should stay **closed for modification** (Open/Closed).
- The vendor class can't be changed: **no source code**, a licence or contract, or other systems depend on it.

The parts

"identify the participants" is asked directly

PART	JOB	BARCODE SCANNER (QUIZ 3)
Client	The existing code. Uses the Target interface only.	`LibrarySystem`
Target	The interface the client expects.	`BarcodeScanner` with `scanBook()`
Adaptee	The existing class with the useful behaviour but the wrong interface.	`NewBarcodeScanner` with `readBarcode()`
Adapter	Implements Target, holds an Adaptee, translates each call.	`ScannerAdapter`

Flow: client calls `scanBook()` → it goes to `ScannerAdapter` → the adapter calls `NewBarcodeScanner.readBarcode()` → the result comes back. Only the adapter knows the adaptee exists.

Three kinds of adapter

use the object adapter unless asked

	OBJECT ADAPTER (DEFAULT)	CLASS ADAPTER	TWO-WAY ADAPTER
How	implements Target, **holds** an Adaptee field	implements Target **and extends** Adaptee	implements **both** old and new interfaces
Good	works with any subclass of the adaptee; loose coupling	no extra object; can override adaptee methods	old and new clients use it during a migration
Bad	one extra object and call	Java has single inheritance; fails if the adaptee is final	more complex, two translations to keep right

Asked before: `Quiz 3` Q1–Q4 barcode scanner · `Quiz 1` Q5 legacy parking payment machine · `2022` Q1.3 third-party travel web services (10m) · `2023` Q3.3 event websites from different vendors (5m). **All on the answers pages.** Java examples you can mention: `InputStreamReader`, `Arrays.asList()`.

04 Adapter: write

Copy in order. Swap the scanner names for the classes in your question.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

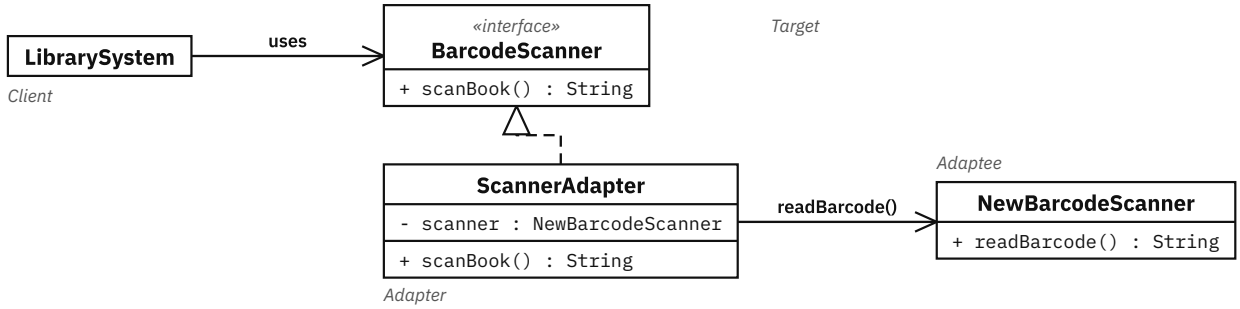
> I would use the **Adapter** pattern, which is a structural pattern. It **converts the interface of a class into another interface the client expects**, so classes with incompatible interfaces can work together.

Reasons: (1) the new scanner is **reused without modifying it** (it is vendor code); (2) the **hundreds of existing `scanBook()` calls stay unchanged**, so there is no risk to tested code (Open/Closed); (3) the client depends only on the Target interface, so **another scanner later is just another adapter**.

Mapping: Client = **LibrarySystem** · Target = **BarcodeScanner** (`scanBook()`) · Adaptee = **NewBarcodeScanner** (`readBarcode()`) · Adapter = **ScannerAdapter**.

W2 Class diagram

the adapter implements the target and points to the adaptee



W3 · JAVA

```java
// 1. TARGET: what the library code expects
public interface BarcodeScanner {
    String scanBook();
}
// 2. ADAPTEE: vendor class, we can't change it
public class NewBarcodeScanner {
    public String readBarcode() {
        return "978-0134685991";
    }
}
```

```java
// 3. ADAPTER: is a Target, wraps Adaptee
public class ScannerAdapter
        implements BarcodeScanner {
    private final NewBarcodeScanner scanner;
    public ScannerAdapter(NewBarcodeScanner s) {
        this.scanner = s;
    }
    public String scanBook() {
        // translate the call
        return scanner.readBarcode();
    }
}
// 4. CLIENT: only this line changes
BarcodeScanner sc =
    new ScannerAdapter(new NewBarcodeScanner());
// all the old calls stay the same
String isbn = sc.scanBook();
```

W4 Same pattern, legacy payment machine

Quiz 1 Q5

Target = `PaymentProcessor.pay(amount)`, the interface the parking system expects. Adaptee = the vendor's `LegacyPaymentMachine`. `LegacyPaymentAdapter` converts rupees → cents and the result code → true/false. The PaymentDue state only calls `processor.pay()`.

```java
class LegacyPaymentAdapter
      implements PaymentProcessor {
  private LegacyPaymentMachine m =
      new LegacyPaymentMachine();
  public boolean pay(double amount) {
    int cents = (int) (amount * 100);
    return m.makeTransaction(cents) == 0;
  }
}
```

04 Adapter: past questions answered

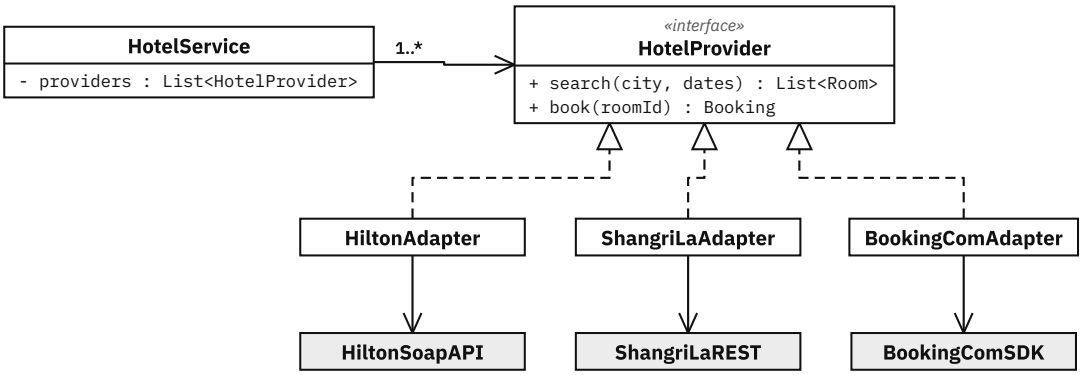
Same pattern every time: one interface your system owns, one adapter per outside system.

STRUCTURAL
ANSWERS

Third-party travel web services

2022 Q1.3, 10m

Pattern: Adapter, one per provider. `HotelService` talks to one interface it owns, `HotelProvider`. `HiltonAdapter`, `ShangriLaAdapter` and so on each translate that interface into the provider's own SOAP / REST / SDK calls. Airlines and taxis are done the same way.



1.3.1 CHALLENGE	1.3.2 MECHANISM
Many adapters to write and maintain	A common interface for all of them; pick the adapter from configuration with a Factory or registry.
Different data formats (XML vs JSON) and error codes	One common data object (DTO) and a mapper inside each adapter; unit-test each adapter.
A provider changes its API version	Only that one adapter changes; version the adapters.
A provider is slow or down	Timeouts, retries, caching, and fall back to another provider.

Event websites from different vendors

2023 Q3.3, 5m

The restaurant owns one interface, `EventSource` with `getEvents()`, returning a common `Event` object. Each website gets its own adapter (`RssEventAdapter`, `JsonApiEventAdapter`, `HtmlScraperAdapter`) that converts that site's format. The restaurant site loops over a list of `EventSource`. A new vendor means a new adapter and nothing else changes.

A **Facade** (`EventFeed`) can then combine all sources into one list.

W5 Challenges → fix

- **Many adapters** (one per vendor) → common Target, created by a Factory.
- **Format translation** → a mapper inside the adapter; unit-test it.
- **Vendor API changes** → only that adapter changes.
- **Adaptee is missing a feature** → the adapter gives a default or throws a clear exception.
- **Extra layer** → tiny overhead; worth it for stable legacy code.

Adapter vs Facade vs Proxy

	ADAPTER	FACADE	PROXY
Goal	convert an interface	simplify a subsystem	control access
Interface	different from the adaptee's	new and simpler	same as the real object
Wraps	usually one object	many objects	one object

05 Proxy

A stand-in with the same interface as the real object. It decides if, when and how the real object is reached.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

load only a couple of pages rather than the entire document · lazy loading · users should not access these objects directly · access controlled through permission checks · restricted archive · different authorization options / levels · cache results · remote service · log every access

The idea

Sometimes the client shouldn't reach the real object straight away. It may be too heavy to load, the user may not be allowed, or it may be on another server.

A proxy has **exactly the same interface**, so the client can't tell the difference. The proxy does its check first (permission, cache, lazy load, log) and only then forwards the call to the real object. Like a security guard at a door: same building, but you go through the guard.

How a call goes through

1. The client calls `getPage(5)` on the Subject (it's really the proxy).
2. The proxy checks first: access, cache, validation, log.
3. If allowed, it creates the real object if needed and forwards the call.
4. The real object does the work and returns.
5. The proxy can cache or log the result, then returns it.

The parts

PART	JOB	LMS DOCUMENT (2022, 2023)
Subject «interface»	The interface both the proxy and the real object implement	`Document` with `getPage(n)`
RealSubject	Does the real, expensive or protected work	`LargeDocument`
Proxy	Same interface, holds a reference to the real one, checks, then forwards	`DocumentProxy`
Client	Uses the Subject and never knows which one it has	`LMSReader`

Types of proxy

name the type in your answer

Virtual	Delays creating a heavy object until it is needed (lazy loading). *LMS pages, high-res images, Google Drive thumbnails.*
Protection	Checks the user's role or permission before allowing the call. *Rare books, hospital records, admin pages, bank withdrawals.*
Remote	A local object that stands for one on another server. *Mobile banking app calling the bank API; an API gateway.*
Cache / smart	Keeps results so repeat calls are fast; can count references. *CDN, search results.*
Logging	Records every access for auditing. *Who opened which record and when.*

Asked before: 2022 Q4.4 and 2023 Q4.4 load only a couple of pages (same question twice) · 2025 Q3.1(c), 3.5 rare books in a restricted archive · 2023 Q4.5, 2024 Q4.5 authorization levels · 2023 Q3.4 logging.

05 Proxy: write

Virtual proxy for the LMS question. The protection proxy is on the answers page.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

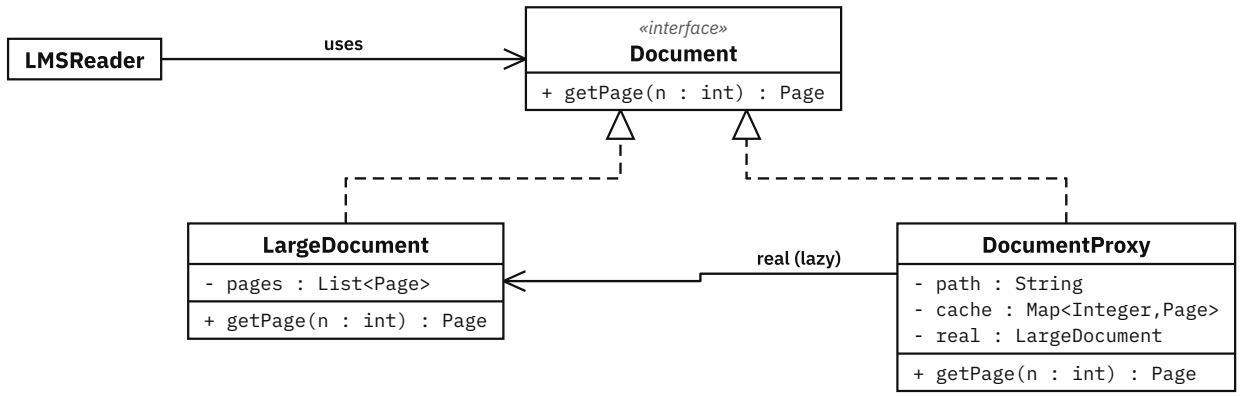
> I would use the **Proxy** pattern (a **virtual proxy**), which is a structural pattern. It provides a **surrogate or placeholder** for another object to **control access** to it.

Reasons: (1) only the pages the user is reading are loaded, so the document **opens fast and uses little memory and bandwidth**; (2) the proxy has the **same interface** as the real document, so the reader code doesn't change; (3) loaded pages can be **cached** so going back is instant.

Mapping: Subject = **Document** «interface» · RealSubject = **LargeDocument** · Proxy = **DocumentProxy** · Client = **LMSReader**.

W2 Class diagram

both implement the Subject; the proxy points to the real subject



W3 · JAVA

```java
// 1. SUBJECT
public interface Document {
    Page getPage(int n);
}

// 2. REAL SUBJECT: expensive to load
public class LargeDocument
        implements Document {
    private List<Page> pages;
    public LargeDocument(String path) {
        pages = Storage.loadAll(path); // slow
    }
    public Page getPage(int n) {
        return pages.get(n);
    }
}

// 4. CLIENT: can't tell it's a proxy
Document doc = new DocumentProxy("notes.pdf");
doc.getPage(1);  // loads page 1 only
doc.getPage(1);  // from cache
```

```java
// 3. PROXY: same interface, lazy
public class DocumentProxy
        implements Document {
    private final String path;
    private final Map<Integer, Page> cache =
        new HashMap<>();

    public DocumentProxy(String path) {
        this.path = path;  // nothing loaded
    }

    public Page getPage(int n) {
        if (!cache.containsKey(n)) {
            // load just this one page
            Page p = Storage.loadPage(path, n);
            cache.put(n, p);
        }
        return cache.get(n);
    }
}
```

Lecture version of the same idea: `ProxyImage` keeps `realImage = null` and only does `new RealImage(file)` the first time `display()` is called.

05 Proxy: past questions answered

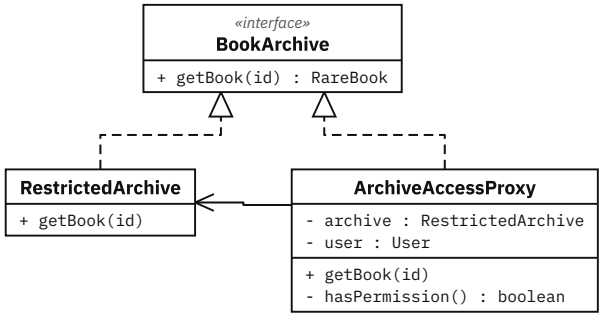
Protection proxy for rare books and authorization, plus the challenges.

STRUCTURAL
ANSWERS

Rare books in a restricted archive

2025 Q3.1(c), 3.5, 5m

Protection Proxy. `ArchiveAccessProxy` and `RestrictedArchive` both implement `BookArchive`. The user only ever gets the proxy. Its `getBook(id)` first checks the user's role and permission (for example, a researcher with an approved request and a valid membership), logs the access, and only then forwards to the real archive. Otherwise it throws `AccessDeniedException`. **Example:** a student asking for an 1820 manuscript is refused; an approved researcher gets it, and the access is logged.




```
public RareBook getBook(String id) {
  if (!user.hasRole("RESEARCHER")
      || !user.isApproved(id))
    throw new
      AccessDeniedException("No permission");
  AuditLog.record(user, id);    // log it
  return archive.getBook(id);   // forward
}
```

Different authorization options

2023 Q4.5 · **2024** Q4.5

Put a **protection proxy** in front of each protected service (`AdminServiceProxy`, `ReportServiceProxy`). It checks the user's role or permissions before forwarding. The services themselves stay free of security code.

If authorization is a series of checks (login → role → permission → 2FA), add a **Chain of Responsibility** (tab 08).

Implementing logging

2023 Q3.4, 5m

Two valid answers. Say which one you mean:

- **Singleton** for the logger itself: one shared logger writing to one file (tab 01).
- **Logging proxy**: to add logging around existing services without changing them, a proxy logs before and after each call and forwards it.

W5 Challenges → fix

- **Extra layer, slower response** → don't use a proxy for cheap, public objects.
- **Stale cache** → set an expiry time, or clear the cache when data changes.
- **Two threads create the real object** → synchronize the lazy-creation check.
- **Proxy becomes a bottleneck / single point of failure** → keep it stateless and run several copies (HA).
- **Interface changes** → proxy and real object must be updated together.

Proxy or not?

Hospital login, online exam	**yes**, protection
Image editor, high-res photos	**yes**, virtual
CDN, Netflix streaming	**yes**, cache + protection
Mobile banking app	**yes**, remote
Calculator, static menu	**no**, nothing to control or delay

Mistakes: giving the proxy a different interface (that's Adapter) · arrow from RealSubject to Proxy · client creating the RealSubject itself.

Slide quiz: true or false?

Proxy and RealSubject are both subtypes of Subject	**True**
Polymorphism comes from both implementing Subject	**True**
RealSubject holds a reference to the Proxy	**False**, it's the other way

Don't mix it up

- **vs Decorator**: both keep the interface. Proxy **controls** access (usually one layer, may create the real object itself). Decorator **adds** features and is stacked.
- **vs Adapter**: Adapter **changes** the interface; Proxy keeps it.

06 Decorator

Add optional extras to an object by wrapping it, layer by layer, instead of making a subclass for every combination.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

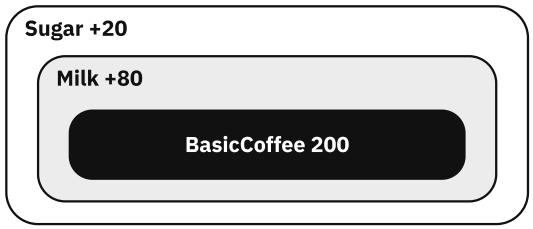
optional add-ons / extras / toppings / ingredients · without creating subclasses for every combination · price depends on the base type and the selected additions · instead of modifying the class directly · students may optionally add · calculate the price dynamically

The idea

A coffee can have milk, sugar and chocolate in any mix. With subclasses you'd need `CoffeeWithMilkAndSugar` and so on: 2^n classes (3 extras → 8, 10 → 1,024). That's **class explosion**.

Instead, each extra is a small wrapper. It looks like a Coffee, holds the coffee inside, adds its own price and text, and passes the call inwards. Stack wrappers at runtime to get any combination.

How a call runs



`getCost()` on Sugar asks Milk, which asks BasicCoffee. The answer comes back out: 200 → 280 → **300**.

The four parts, renamed for each question

naming the parts = marks

	COMPONENT «INTERFACE»	CONCRETE COMPONENT (THE BASE)	CONCRETE DECORATORS (THE EXTRAS)
Coffee 2024	`Coffee`	`BasicCoffee`	Milk, Sugar, Chocolate
Kiosk 2025	`CheckoutItem`	`GroceryItem(name, price)`	GiftWrap, EcoBag, ExpressPacking
Profile Q5	`UserProfile`	`BasicUserProfile`	PremiumBadge, TwoFactorAuth, ProfileTheme
Library 2025	`BorrowTransaction`	`BasicBorrow`	ExtendedPeriod, SmsNotification, EmailReceipt
Cake 2022	`Cake`	`PlainCake`	Fondant, Colour (each adds price *and* delivery days)
Course Q6	`Course`	`ProgrammingCourse`	AIAssistant, PremiumMaterials, MentorSupport

The fourth part, the abstract **Decorator**, is always `<Component>Decorator` (CoffeeDecorator, AddOnDecorator). It **is a** Component (implements it) **and has a** Component (a field holding the wrapped object).

Don't mix it up

- **vs Proxy**: same interface, but Proxy controls access (one layer). Decorator adds features (many layers).
- **vs Adapter**: Adapter changes the interface. Decorator keeps it.
- **vs inheritance**: inheritance fixes combinations at compile time. Decorator builds them at runtime.

Mistakes that lose marks

- Decorator doesn't implement the Component.
- Decorator makes its own base instead of taking it in the constructor.
- Not calling the wrapped object, so earlier extras are lost.
- Inheritance drawn instead of the has-a diamond.

Asked before: 2024 coffee (20m) · 2025 kiosk (11m) · 2023 coffee flavours · Quiz 5 profile (18m) · 2022 cake · 2025 library · Quiz 6 course. **Use the method names the question gives you.**

06 Decorator: write

Copy in order. Swap Coffee and the extras for the names in your question (table on the previous page).

STRUCTURAL
WRITE

W1 Name it and say why

2–6 marks

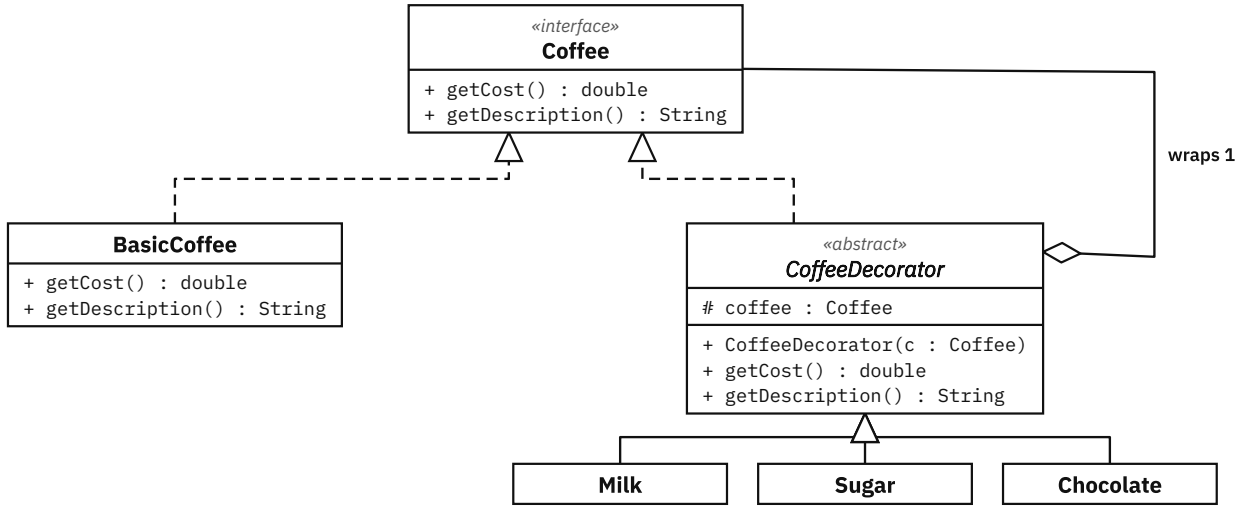
> I would use the **Decorator** pattern, which is a structural pattern. It **attaches additional responsibilities to an object dynamically** by wrapping it, which is a flexible alternative to subclassing.

Reasons: (1) **no class explosion**: n add-ons would need 2^n subclasses, here it's one class per add-on; (2) **runtime flexibility**: the customer picks any mix of add-ons while ordering; (3) **Open/Closed**: a new add-on is a new class, and the base coffee and existing add-ons don't change; (4) each decorator has **one responsibility**.

Mapping: Component = **Coffee** «interface» · concrete component = **BasicCoffee** · decorator = **CoffeeDecorator** «abstract» · concrete decorators = **Milk, Sugar, Chocolate**.

W2 Class diagram

the diamond sits on the decorator: it has a Coffee inside



W3 · JAVA (THIS IS WHAT 2024 Q1.2–1.3 AND 2025 Q1.2–1.3 ASKED FOR)

```java
// 1. COMPONENT: "define a Coffee interface"
public interface Coffee {
    double getCost();
    String getDescription();
}

// 2. CONCRETE COMPONENT: "create BasicCoffee"
public class BasicCoffee implements Coffee {
    public double getCost() { return 200.0; }
    public String getDescription() {
        return "Basic coffee"; }
}

// 5. CLIENT: wrap at runtime
Coffee c = new BasicCoffee();
c = new Milk(c);
c = new Sugar(c);
c.getDescription(); // Basic coffee, Milk, Sugar
c.getCost();        // 200 + 80 + 20 = 300.0
```

```java
// 3. ABSTRACT DECORATOR: is-a + has-a
public abstract class CoffeeDecorator
        implements Coffee {
    protected Coffee coffee;      // wrapped
    public CoffeeDecorator(Coffee c) {
        this.coffee = c; }
    public double getCost() {
        return coffee.getCost(); }
    public String getDescription() {
        return coffee.getDescription(); }
}

// 4. CONCRETE DECORATOR: add, then pass on
public class Milk extends CoffeeDecorator {
    public Milk(Coffee c) { super(c); }
    public double getCost() {
        return coffee.getCost() + 80; }
    public String getDescription() {
        return coffee.getDescription()
               + ", Milk"; }
}
// Sugar (+20), Chocolate (+120): same shape
```

Decorator: past questions answered

The follow-up parts they add after the basic code.

W4 "If there are different profits for each coffee, how would you extend the design?"

2024 Q1.4, 5m

Option A: add `getProfit()` to the `Coffee` interface. Each base coffee returns its own margin, and each decorator returns `coffee.getProfit() + addOnProfit`. Selling price = `getCost() + getProfit()`.

Option B (better, keeps Open/Closed): a new `ProfitMargin` decorator wraps the finished drink and adds a percentage. No existing class changes. Different coffees get different rates: `new ProfitMargin(irish, 0.30)`.

```
public class ProfitMargin
        extends CoffeeDecorator {
    private double rate;
    public ProfitMargin(Coffee c, double r) {
        super(c);
        rate = r;
    }
    public double getCost() {
        return coffee.getCost() * (1 + rate);
    }
}
```

Self-checkout kiosk: CheckoutItem and GroceryItem

2025 Q1.2–1.3, 10m

```
public interface CheckoutItem {
    double getCost();
    String getDescription();
}
public class GroceryItem
        implements CheckoutItem {
    private String name;
    private double price;
    public GroceryItem(String n, double p) {
        name = n; price = p;
    }
    public double getCost() { return price; }
    public String getDescription() {
        return name; }
}
```

```
public class GiftWrap
        extends AddOnDecorator {
    public GiftWrap(CheckoutItem i) {
        super(i); }
    public double getCost() {
        return item.getCost() + 150; }
    public String getDescription() {
        return item.getDescription()
               + " + gift wrap"; }
}
// usage
CheckoutItem it = new GroceryItem(
        "Rice 5kg", 1200);
it = new EcoBag(it);
it = new GiftWrap(it);
```

User profile: wrapping and real-world challenges

Quiz 5, 3m + 4m

```
UserProfile p = new BasicUserProfile(
    "Nimal", "nimal@mail.com", "n.png");
p = new PremiumBadgeDecorator(p);
p = new TwoFactorAuthDecorator(
        p, "0771234567");
p = new ProfileThemeDecorator(p, "Dark");
p.getDetails();
// [PREMIUM] Nimal (nimal@mail.com)
//   | theme: Dark
```

Code complexity	Many small classes; hard to see one object's full feature set.
Debugging	A call passes through many wrappers; the bug can be in any layer.
Performance	An extra object and call per layer; small, but it grows.
Readability	`new A(new B(new C(x)))` is hard to read, and order changes the result.

W5 Challenges → fix

- **Many small classes** → one job per decorator; a builder assembles common combinations.
- **Debugging** → log in each decorator; test each one on its own.
- **Order matters** (discount then tax ≠ tax then discount) → fix the wrapping order in the builder.
- **Performance** → negligible for short chains; cache the total.

07 Template Method

The parent class fixes the steps and their order. Each subclass fills in only the steps that are different for it.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

all types follow the same overall process · however, step 2 is different for each type · standardise the common workflow while allowing custom behaviour · every level must follow the same procedure · enforce the procedure · salary components with a few deviations · copy-pasted algorithm with small differences

The idea

Card, PayPal and cash-on-delivery payments all do the same three things in the same order: validate, process, send a receipt. Only "process" really differs.

So you write the order once, in a **final** method in an abstract parent class. The common steps are written there too. The step that differs is left **abstract**, and each subclass writes only that step. Like a recipe card: same steps for every cake, only the filling changes.

This is the **Hollywood principle**: "don't call us, we'll call you". The parent calls the child's steps.

The four kinds of method

know these

KIND	MODIFIER	JOB
Template method	`public final`	Calls the steps in a fixed order. Final, so no subclass can change the order.
Common step	`private` or concrete	Written once, shared by all.
Primitive step	`protected abstract`	Each subclass must write it.
Hook	`protected` with a default	Subclass may override it if it wants (e.g. `wantsEmail()` returns true).

OOP ideas used

"explain how abstract classes are used"

Inheritance (subclasses extend the parent) · **abstract class and abstract methods** (the parent can't be created on its own and forces subclasses to write the varying steps) · **method overriding** · **final** (locks the order) · **access modifiers** (protected steps, private common steps) · **polymorphism** (the parent calls whichever subclass version exists).

Why an abstract class and not an interface? The base has to hold real code (the common steps and the fixed order) as well as abstract steps. An interface can't enforce the order or share that code.

Don't mix it up

- **vs Strategy**: Template varies **some steps** using inheritance, fixed at compile time. Strategy swaps the **whole algorithm** as an object at runtime.
- **vs Factory Method**: a factory method is often just one step inside a template method.
- **vs Chain**: Template is one object running fixed steps. Chain passes a request between objects.

Mistakes that lose marks

- Template method not marked `final`.
- Making every step abstract (then nothing is shared).
- Using an interface for the base class.
- The client calling the steps one by one.
- Drawing dashed (implements) arrows. Subclasses **extend**, so solid lines.

Asked before: 2025 Q4.1(a), 4.2, 3.3 payment workflow (abstract PaymentProcessor + CreditCard class) · 2022 Q2 and 2023 Q1 salary calculation (25m each) · 2024 Q4.2, 4.4 checkout steps · Quiz 6 course completion · Quiz 7 Q4 support procedure. **All on the answers pages.**

07 Template Method: write

The payment workflow from 2025 Q4. Swap the steps for the ones in your question.

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

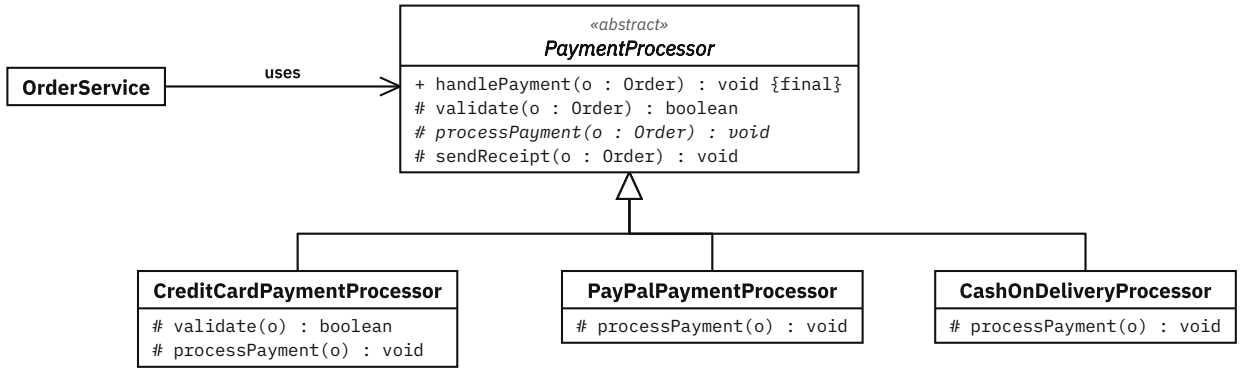
> I would use the **Template Method** pattern, which is a behavioural pattern. It **defines the skeleton of an algorithm in a superclass** and lets subclasses **redefine certain steps** without changing the algorithm's structure.

Reasons: (1) every payment type follows the **same order** (validate → process → receipt) and the final template method **enforces** it; (2) the common steps are **written once**, so there is no duplicated code; (3) a new payment type only writes `processPayment()`, so it is **easy to extend** (Open/Closed) and behaviour is consistent across types.

Mapping: abstract class = **PaymentProcessor** · template method = `handlePayment()` (final) · common steps = `validate()`, `sendReceipt()` · varying step = `processPayment()` · concrete classes = **CreditCard, PayPal, CashOnDelivery** processors.

W2 Class diagram

italic = abstract · {final} on the template method · solid arrows (extends)



W3 · JAVA (2025 Q4.2: ABSTRACT PAYMENTPROCESSOR · Q3.3: CREDITCARDPAYMENTPROCESSOR)

```java
public abstract class PaymentProcessor {

  // TEMPLATE METHOD: fixed order, final
  public final void handlePayment(Order o) {
    if (!validate(o)) {
      System.out.println("Invalid details");
      return;
    }
    processPayment(o);
    sendReceipt(o);
  }

  // common step (subclass may extend it)
  protected boolean validate(Order o) {
    return o.getAmount() > 0
        && o.getCustomer() != null;
  }
  // the step that differs
  protected abstract void
      processPayment(Order o);

  // common step
  protected void sendReceipt(Order o) {
    System.out.println("Receipt sent");
  }
}
```

```java
public class CreditCardPaymentProcessor
        extends PaymentProcessor {

  // adds a card check to the common one
  protected boolean validate(Order o) {
    return super.validate(o)
        && CardValidator.isValid(o.getCardNo());
  }

  // only the varying step is written here
  protected void processPayment(Order o) {
    System.out.println("Charging card for "
        + o.getAmount());
    BankGateway.charge(o.getCardNo(),
                       o.getAmount());
  }
}

// CLIENT
PaymentProcessor p =
    new CreditCardPaymentProcessor();
p.handlePayment(order);
// validate → process → receipt
```

07 Template Method: salary questions

2022 Q2 and 2023 Q1 were both 25-mark payroll questions. One design answers both.

BEHAVIOURAL
ANSWERS

Which pattern, and why

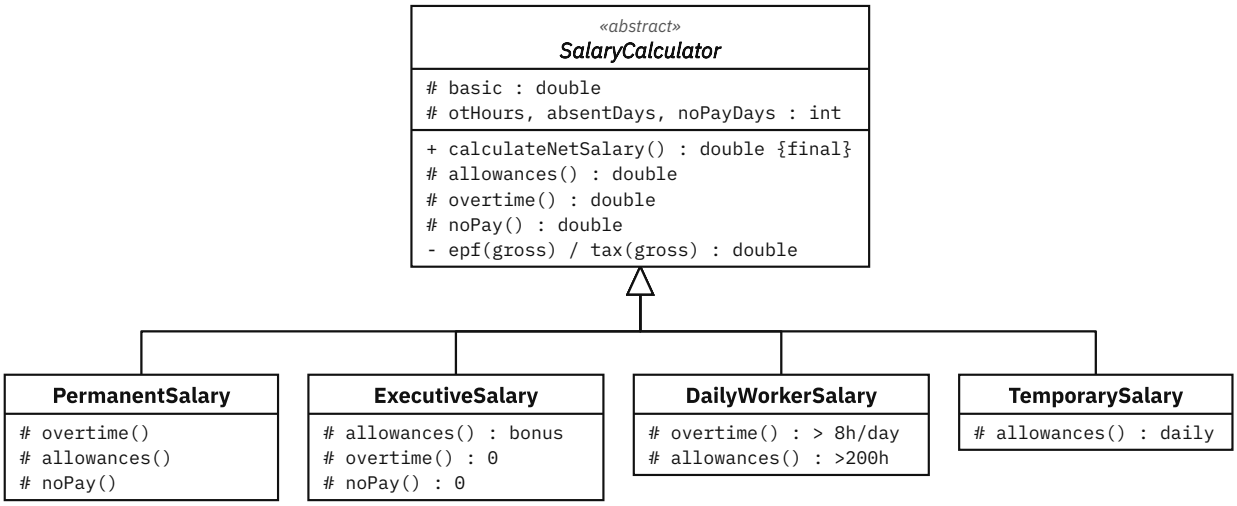
2022 Q2.2 6m · **2023** Q1.2 5m

Template Method. Every employee's salary is worked out in the same order: gross → EPF → deductions → net. The EPF, tax and net formulas are the same for everyone. Only some components differ (overtime, allowances, no-pay). So `calculateNetSalary()` is a final template method in an abstract `SalaryCalculator`, the common formulas are written there once, and each employee type overrides only its own components. 2023's deviation is easy: `ExecutiveSalary` overrides `overtime()` and `noPay()` to return 0. The 2023 formulas map straight onto the hooks: `overtime()` = basic × OT hours / (30 × 8), `noPay()` = basic × days / 30, attendance allowance in `allowances()` (absent 0 → 75%, 1–3 → 50%, more than 3 → none).

Class diagram

· 2022 Q2.3 5m

hooks return 0 by default; subclasses override only what differs




```
public abstract class SalaryCalculator {
  protected double basic;

  public final double calculateNetSalary() {
    double gross = basic + allowances()
                 + overtime();
    double epf = gross * Config.get("epf");
    double tax = Math.max(0,
                 (gross - 100000) * 0.06);
    return gross - (epf + noPay() + tax);
  }
  // hooks: default 0, override if needed
  protected double allowances() { return 0; }
  protected double overtime()   { return 0; }
  protected double noPay()      { return 0; }
}
```

Challenges → mechanism

Q2.4–2.5 / Q1.3–1.4

Rigid order: one type needs a different sequence	Hooks for optional steps; if the order truly differs, use Strategy.
Many subclasses as types grow	Shared middle classes (e.g. `HourlyPaid`); keep steps small.
Rates change (EPF, tax)	Read them from a Singleton config (tab 01 answers).
A subclass overrides a common step	Make the template final and common steps private/final.

"Code duplication can be achieved from this pattern." Agree?

2022 Q2.6, 4m

No, as worded. Template Method **avoids** duplication: the common formulas (EPF 8%, tax, net) exist once in the superclass, and subclasses contain only their differences. A little duplication can remain if two subclasses override a step the same way; the fix is to move that code up into the parent or into a shared middle class.

07 Template Method: more answers

Course completion, checkout steps, the support procedure, and the challenges.

BEHAVIOURAL
ANSWERS

Online course completion

Quiz 6, 2 + 8 + 3 + 4m

Why Template Method: every course follows the same completion order, and it must be enforced, but two steps differ per course (programming: coding challenge marked by automated tests; design: portfolio marked by an instructor with a rubric). The abstract class fixes the order in a final `completeCourse()`, writes the common steps once, and leaves the two varying steps abstract.

```
public abstract class OnlineCourse {
  public final void completeCourse() {
    if (!validateRegistration()) return;
    deliverFinalAssessment();
    double score = evaluatePerformance();
    generateCertificate(score);
  }
  private boolean validateRegistration() {
    return true; } // common
  protected abstract void
      deliverFinalAssessment();
  protected abstract double
      evaluatePerformance();
  private void generateCertificate(double s) {
    if (s >= 50)
      System.out.println("Certificate");
  }
}
```

```
public class ProgrammingCourse
        extends OnlineCourse {
  protected void deliverFinalAssessment() {
    System.out.println("Coding test"); }
  protected double evaluatePerformance() {
    return AutoTests.run(studentId); }
}
public class DesignCourse
        extends OnlineCourse {
  protected void deliverFinalAssessment() {
    System.out.println("Submit portfolio"); }
  protected double evaluatePerformance() {
    return Instructor.rubric(studentId); }
}
// Q5 optional extras → Decorator
```

E-commerce checkout

2024 Q4.2, 4.4, 10m

Template Method. `CheckoutProcess` has a final `checkout()`: validate cart → process payment → arrange delivery → send confirmation. Validate cart and send confirmation are common; payment and delivery (domestic / international) vary.

Use of abstract classes: `CheckoutProcess` is abstract because it holds real code (common steps and the fixed order) plus abstract steps. It can't be instantiated; subclasses supply the varying parts.

4.3 exceptions → Chain (tab 08) · 4.5 authorization → Proxy (tab 05)

Support procedure

Quiz 7 Q4, 4m

Every support level must log the request → attempt to solve → log the resolution. Put a **final** `handle()` in the abstract `SupportHandler` that does exactly that and calls an abstract `attemptToSolve()`. L1, L2, L3 and Specialist only write `attemptToSolve()`, so no level can skip the logging. This is Template Method inside a Chain of Responsibility (code in tab 08).

W5 Challenges → fix

from the slides

CHALLENGE	HOW TO HANDLE IT
The client **can't change the sequence** of steps	That's the point; if order must vary per request, use Builder or Strategy instead. Use hooks for optional steps.
A subclass may **override a parent method** it shouldn't	Template method `final`; common steps `private` or `final`; only primitives `protected abstract`.
More subclasses → code spread out, harder to maintain	Keep steps small; shared middle abstract classes; document each step.
Rigid: changing the skeleton affects every subclass	Design the skeleton carefully; add new optional steps as hooks with default bodies.

08 Chain of Responsibility

A request is passed along a line of handlers. Each one either deals with it or hands it to the next.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

several levels of support / escalation · approval limits: team leader, manager, director, CEO · a sequence of checks that must occur in order and be easy to extend · multiple sub-filters · handle different priorities · exception handling · multiple handlers hard-coded in sequence

The idea

Without the pattern, the sender has a long if-else: "if it's a password issue send it to L1, else if it's an install issue send it to L2...". The sender knows every handler, and adding a level means editing it.

With the chain, each handler only knows the **next** one. The sender gives the request to the first handler and doesn't care who finally deals with it. Like a hospital: nurse → doctor → specialist, each passing on what they can't treat. Java's try/catch works the same way.

Two ways a chain runs

1. Escalation: the first handler that **can** handle the request does, and the chain stops. *Support levels, expense approval, exceptions by priority.*

2. Pipeline of checks: every handler checks the request; any failure stops it, and passing all means accepted. *Membership → availability → fines; inventory → fraud → delivery area; spam sub-filters.*

The chain ends when a handler deals with the request, or when the end is reached. Add a **fallback handler** at the end so nothing goes unhandled.

The parts

PART	JOB	SUPPORT DESK (QUIZ 7)
Handler «abstract»	Holds a `next` link, has `setNext()` and `handle()`	`SupportHandler`
Concrete handlers	Deal with the request if they can, otherwise pass it on	`L1Support`, `L2Support`, `L3Support`, `SpecialistSupport`
Request	The data being passed (type, level, amount)	`SupportRequest`
Client	Builds the chain once, sends every request to the first handler	`SupportDesk`

How to order the chain

asked as "prioritise"

- **Cost / speed**: cheap, fast checks first, so expensive ones run less often.
- **Rejection rate**: checks that catch the most go early.
- **Severity / security**: dangerous cases first (e.g. a malware attachment).
- **Accuracy**: low false-positive checks before risky ones.
- **Business priority** and **availability** of the handler.

Don't mix it up

- **vs State**: Chain asks "**who** will handle this request?" and the request moves along handlers. State asks "**how** should I behave now?" and the object hands work to its current state.
- **vs Decorator**: decorators all run and add to the result. In a chain, usually one handler takes the request.

Asked before: 2022 Q3 email spam filter (all 25 marks) · 2025 Q3.1(a), 3.4 membership → availability → fines · 2025 Q4.1(b), 4.4 inventory → fraud → delivery area · 2023 Q4.3, 2024 Q4.3 exceptions by priority · Quiz 7 support levels (20m).

08 Chain of Responsibility: write

The support desk from Quiz 7, with the logging procedure enforced (Quiz 7 Q4).

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

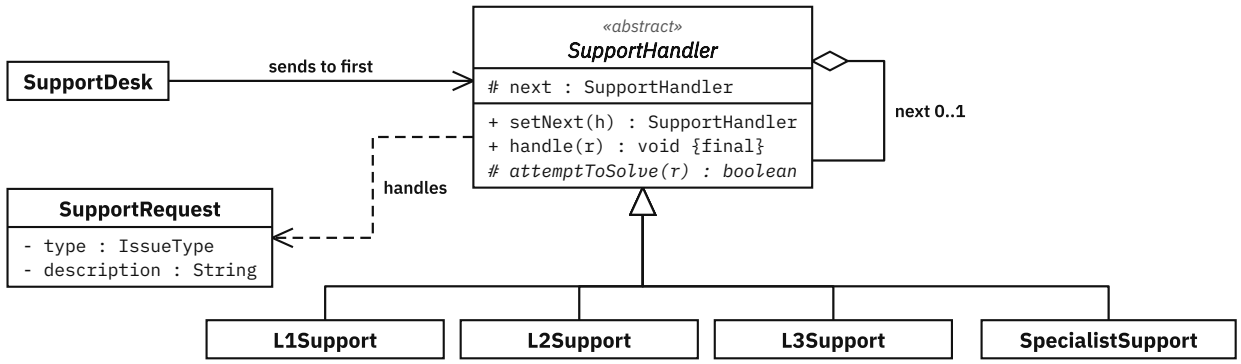
> I would use the **Chain of Responsibility** pattern, which is a behavioural pattern. It **avoids coupling the sender of a request to its receiver** by giving more than one object a chance to handle it. The handlers are chained, and the request is passed along until one handles it.

Reasons: (1) the support desk sends every request to L1 and **doesn't need to know who solves it**; (2) each level has **one responsibility**; (3) levels can be **added, removed or reordered** without changing the client or the other levels (Open/Closed); (4) a fallback at the end means **nothing goes unhandled**.

Mapping: request = **SupportRequest** · handler = **SupportHandler** «abstract» with `next` · concrete handlers = **L1, L2, L3, Specialist** · client = **SupportDesk**.

W2 Class diagram

the "next" link is a self-association on the Handler



W3 · JAVA (QUIZ 7 Q3: A DATABASE CONNECTIVITY REQUEST GOES THROUGH THE CHAIN)

```java
public abstract class SupportHandler {
  protected SupportHandler next;

  public SupportHandler setNext(
          SupportHandler n) {
    this.next = n;
    return n;   // allows chaining
  }

  // fixed procedure (Template Method, Q4)
  public final void handle(SupportRequest r) {
    log("received " + r.getType());
    boolean solved = attemptToSolve(r);
    log(solved ? "RESOLVED" : "escalated");
    if (!solved && next != null) {
      next.handle(r);      // pass it along
    }
  }
  protected abstract boolean
      attemptToSolve(SupportRequest r);

  private void log(String msg) {
    System.out.println(
      getClass().getSimpleName() + ": " + msg);
  }
}
```

```java
public class L1Support extends SupportHandler {
  protected boolean attemptToSolve(
          SupportRequest r) {
    return r.getType() == PASSWORD_RESET
        || r.getType() == ACCOUNT_BLOCKED;
  }
}
// L2Support: INSTALLATION_ERROR
public class L3Support extends SupportHandler {
  protected boolean attemptToSolve(
          SupportRequest r) {
    return r.getType() == DB_CONNECTIVITY
        || r.getType() == PERFORMANCE;
  }
}
// SpecialistSupport: returns true (fallback)

// CLIENT: build once, send to the first
SupportHandler chain = new L1Support();
chain.setNext(new L2Support())
     .setNext(new L3Support())
     .setNext(new SpecialistSupport());
chain.handle(new SupportRequest(
    DB_CONNECTIVITY, "App can't reach MySQL"));
// L1 escalated → L2 escalated → L3 RESOLVED
```

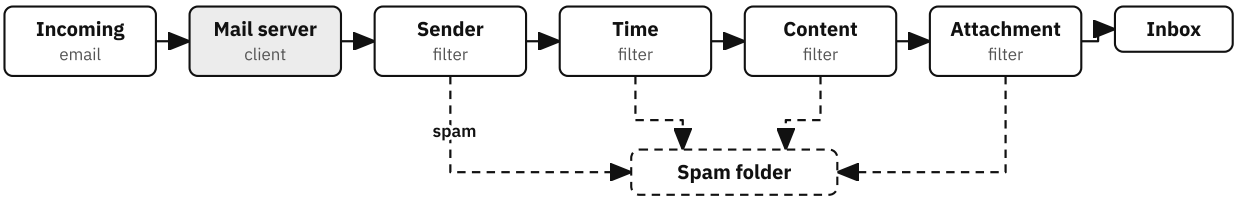
08 Chain: the spam filter question

2022 Q3 was a full 25-mark Chain question. All five parts, answered.

BEHAVIOURAL
ANSWERS

3.1 Diagram and components · 4m

solid = passed, go on · dashed = marked as spam, stop



Components: the **mail server** (client) receives the email and passes it to the first filter. **Sender filter** (blacklist, domain reputation), **time filter** (odd hours, bursts), **content filter** (keywords, links, NLP), **attachment filter** (file types, virus scan). Any filter can mark it SPAM and stop; if all pass, it goes to the inbox.

3.2 Pattern and reasons

5m

Chain of Responsibility. Each sub-filter is a handler with a link to the next one. The email passes Sender → Time → Content → Attachment. Reasons: the mail server is decoupled from the filters (it only knows the first); each filter has one responsibility; filters can be added, removed or reordered without changing the others; the chain can be configured at runtime.

3.3 Prioritising the components

6m

- **Cost and speed**: cheap checks first (sender blacklist, sent time); expensive ones last (content NLP, attachment virus scan).
- **Rejection rate**: the filter that catches the most spam early saves work for the rest.
- **Severity**: a malware attachment may be moved first for security.
- **Accuracy**: filters with few false positives before risky ones.

So a sensible order is: **Sender → Time → Content → Attachment**.

3.4 Adding more sub-components

5m

Write a new class that extends `SpamFilter` (e.g. `LinkReputationFilter`) and link it in with `setNext()`, or list it in a config file that builds the chain. Existing filters and the mail server don't change (Open/Closed).

3.5 High Availability and DR

5m

HA: the filters are stateless, so run the chain on several servers behind a load balancer; a backup handler skips a filter that times out; a message queue so no mail is lost.
DR: blacklists and rules backed up and replicated to a second region, with automatic failover.

08 Chain: validation checks and more

The 2025 "checks in order" questions, exceptions, authorization, and the challenges.

BEHAVIOURAL
ANSWERS

Checks that must run in order

2025 Q4.4 inventory → fraud → delivery · Q3.4 membership → availability → fines, 5m

Chain of Responsibility in the **pipeline** form. Each check is a handler. If it fails, the chain stops and returns the reason ("membership expired"); if it passes, the request goes to the next check. A new check (e.g. `BorrowLimitCheck`) is one new class and one new link.

```
public abstract class Validator {
  private Validator next;
  public Validator linkWith(Validator n) {
    this.next = n;
    return n;
  }
  public boolean validate(Order o) {
    if (!check(o)) return false; // stop
    return next == null || next.validate(o);
  }
  protected abstract boolean check(Order o);
}
```

```
class InventoryCheck extends Validator {
  protected boolean check(Order o) {
    return Stock.has(o.getItems()); }
}
// FraudCheck, DeliveryAreaCheck: same shape

// CLIENT
Validator v = new InventoryCheck();
v.linkWith(new FraudCheck())
 .linkWith(new DeliveryAreaCheck());
if (v.validate(order))
  processor.handlePayment(order);
```

Exceptions with different priorities

2023 Q4.3 · **2024** Q4.3

Chain of handlers ordered by severity: `ValidationErrorHandler` (low: show a message) → `PaymentErrorHandler` (retry or offer another method) → `SystemErrorHandler` (log, alert the admin) → `DefaultHandler` (generic error page). Each one handles the types it knows and passes the rest on, just like nested try/catch blocks.

Expense approval

in-class example

Team leader	up to $1,000
Manager	$1,001 – $5,000
Director	$5,001 – $20,000
CEO	above $20,000

$750 → team leader · $3,500 → manager · $50,000 → CEO. Authorization levels (2024 Q4.5) work the same way.

W5 Challenges → fix

from the slides

CHALLENGE	HOW TO HANDLE IT
No guarantee the request is handled (falls off the end)	A default / fallback handler at the end (Specialist, manual review).
Debugging is tricky	Log at every handler (who received it, who handled it); unit-test each handler.
Long chains are slow; unnecessary handlers run	Order by priority (cheap and likely first); skip handlers using request info; keep the chain short.
Order matters; cycles if mis-linked	Build the chain in one place (builder or config) and check for cycles.

Mistakes: the client calling every handler itself (that's the if-else approach) · forgetting the `next != null` check · no fallback handler · drawing links between concrete handlers instead of the self-association on Handler.

09 State

An object behaves differently depending on which state it is in. Each state gets its own class.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

can move through the following states · operates in different states: Idle, Scanning, Payment, Completed · transitions to · the behaviour depends on its current state · define / draw the state diagram · behaviour changes through flags and conditions

The idea

Press "mute" on a TV remote. If the TV is on, it mutes. If it's off, nothing happens. Same button, different result, because of the **state**.

Without the pattern, every method is a big `if (state == ON) … else if (state == OFF) …`, and it grows with every new state. With State, each state is a class that knows what to do for each action and which state comes next. The main object (the **context**) just says "current state, you handle this".

TV remote state table

lecture example

STATE	EVENT	DOES	NEXT
OFF	on	turn TV on	ON
OFF	off / mute	nothing	OFF
ON	mute	sound off	MUTE
ON	off	turn TV off	OFF
MUTE	on	sound back	ON
MUTE	off	turn TV off	OFF

The parts

PART	JOB	DOCUMENT WORKFLOW (2024 Q3)
Context	Holds the current state, passes every request to it, has `setState()`	`Document`
State «interface»	One method per event; every state must have them all	`DocumentState`
Concrete states	The behaviour for that state; they call `context.setState(next)` to move on	`DraftState`, `ModerationState`, `PublishedState`, `ArchivedState`

Designing it in 5 steps

1. Draw the **state diagram** (states and events).
2. Make the **state table**: state + event → what happens + next state.
3. State **interface** = one method per event.
4. **One class per state** implementing it.
5. The **context delegates**: `state.submit(this)`.

Don't mix it up

- **vs Chain**: State asks "how should I behave now?" and the object delegates to its current state. Chain asks "who handles this?" and the request moves along handlers.
- **vs Strategy**: same structure, but the client picks a strategy and it doesn't change itself. States switch themselves.

Asked before: `2024` Q3 document workflow (state diagram 5, pattern 5, class diagram 10, interfaces 5) · `2025` Q1.1(a), 1.4 kiosk states · `Quiz 1` smart parking (why, state diagram, UML, context code). **Rule of thumb:** long chains of `if (state == …)` mean State.

09 State: write

The document workflow from 2024 Q3 (25 marks). Swap the states for the ones in your question.

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

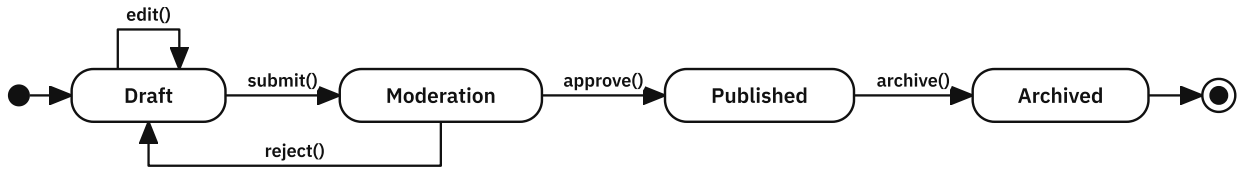
> I would use the **State** pattern, which is a behavioural pattern. It allows an object to **change its behaviour when its internal state changes**; the object appears to change its class.

Reasons: (1) the document has a **fixed set of states** and the same action behaves differently in each (edit works in Draft but is refused in Published); (2) it **removes the if/else on a status field** from every method; (3) each state's rules and **transitions live in one class**, and a new state is just a new class (Open/Closed).

Mapping: context = **Document** · state interface = **DocumentState** · concrete states = **Draft, Moderation, Published, Archived**.

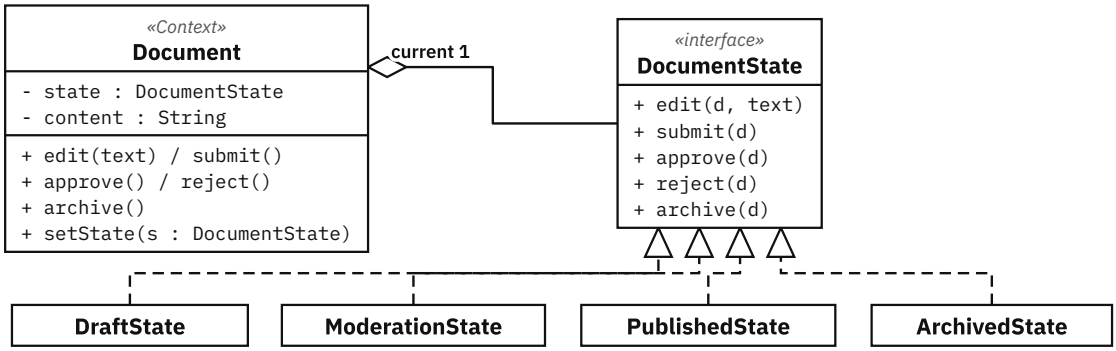
W2a State diagram · 2024 Q3.1, 5m

label every arrow with its event; Archived is final



W2b Class diagram · 2024 Q3.3, 10m

the diamond sits on the context: it has a current state



W3 · JAVA

```java
public interface DocumentState {
  void edit(Document d, String text);
  void submit(Document d);
  void approve(Document d);
  void reject(Document d);
  void archive(Document d);
}
public class Document {          // CONTEXT
  private DocumentState state =
      new DraftState();
  private String content = "";
  void setState(DocumentState s) { state = s; }
  public void edit(String t) {
    state.edit(this, t); }
  public void submit()  { state.submit(this); }
  public void approve() { state.approve(this); }
  public void reject()  { state.reject(this); }
  public void archive() { state.archive(this); }
}
```

```java
public class ModerationState
        implements DocumentState {
  public void approve(Document d) {
    d.setState(new PublishedState()); }
  public void reject(Document d) {
    d.setState(new DraftState()); }
  public void edit(Document d, String t) {
    System.out.println("Under review"); }
  public void submit(Document d) {
    System.out.println("Already submitted"); }
  public void archive(Document d) {
    System.out.println("Publish first"); }
}
// Draft: edit ok, submit → Moderation
// Published: archive → Archived only
// Archived: everything refused
```

09 State: smart parking

Quiz 1, the lecturer's own State question: why, state diagram, UML, context code, then an Adapter twist.

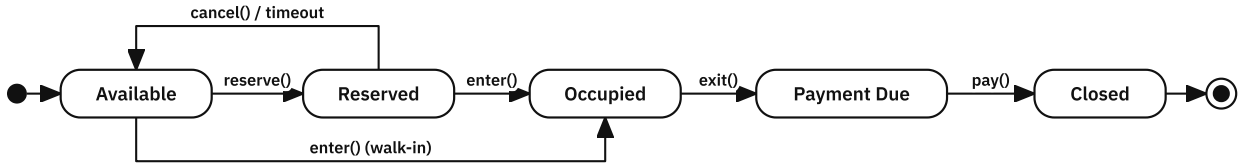
BEHAVIOURAL
ANSWERS

Q1 Why State is appropriate

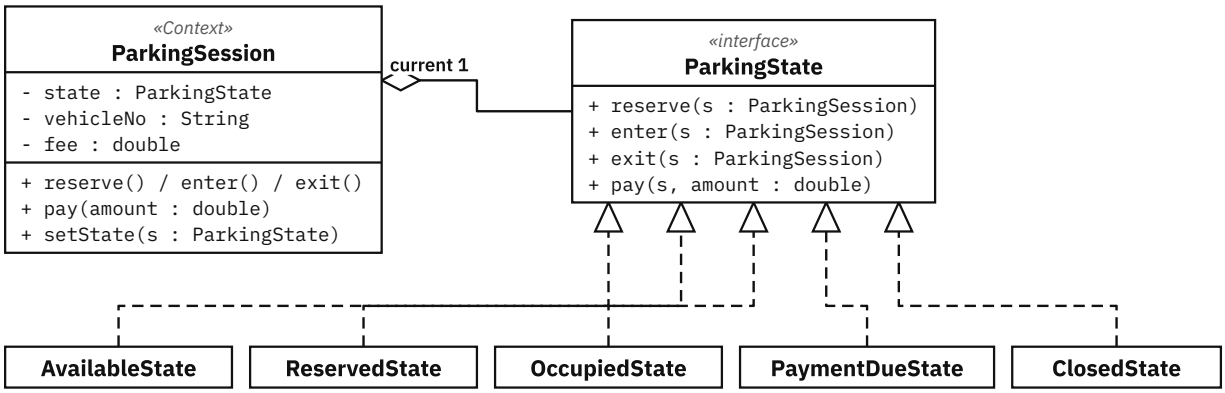
4m

A parking session has a **fixed set of well-defined states** (Available, Reserved, Occupied, Payment Due, Closed), and the **same action behaves differently in each**: `enter()` is allowed when Reserved but refused when Closed, and `pay()` only works in Payment Due. Without the pattern every method needs a growing if/switch on the status. With State, each state's rules live in one class, transitions are explicit, invalid actions are refused cleanly, and a new state (e.g. Overstay) is just a new class.

Q2 State diagram · 4m



Q3 UML class diagram · 6m



Q4 CONTEXT CLASS · 3M

```
public class ParkingSession {
  private ParkingState state =
      new AvailableState();
  private String vehicleNo;
  private double fee;

  public void setState(ParkingState s) {
    state = s; }
  public void reserve() { state.reserve(this); }
  public void enter()   { state.enter(this); }
  public void exit()    { state.exit(this); }
  public void pay(double amt) {
    state.pay(this, amt); }
}
```

Q5 Legacy payment machine

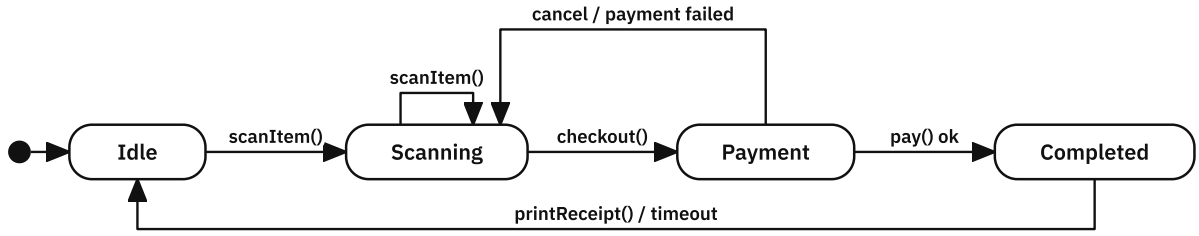
The vendor's machine can't be modified, so use an **Adapter**. `LegacyPaymentAdapter` implements the `PaymentProcessor` interface the system expects and calls the vendor's method inside. `PaymentDueState.pay()` calls `processor.pay(amount)`; if it succeeds it moves the session to Closed. Code: tab 04, W4.

09 State: kiosk and interfaces

2025 Q1 kiosk states with an example transition, 2024 Q3.4 on interfaces, and the challenges.

BEHAVIOURAL
ANSWERS

Self-checkout kiosk states · 2025 Q1.1(a), 1.4, 5m



How to incorporate the states, with an example transition

State pattern. `Kiosk` is the context and holds a `KioskState`. `IdleState`, `ScanningState`, `PaymentState` and `CompletedState` implement `scanItem()`, `checkout()`, `pay()` and `cancel()`.

Example: in `ScanningState`, pressing Checkout adds up the decorated items' `getCost()`, adds tax with `TaxCalculator.getInstance()` (Singleton) and moves to `PaymentState`. In `IdleState`, `pay()` is refused: "scan items first".

```
public class ScanningState
        implements KioskState {
  public void scanItem(Kiosk k,
                       CheckoutItem i) {
    k.addItem(i); }        // stay here
  public void checkout(Kiosk k) {
    double total = TaxCalculator
        .getInstance()
        .applyTax(k.subtotal());
    k.setTotal(total);
    k.setState(new PaymentState());
  }
  public void pay(Kiosk k, double a) {
    System.out.println("Checkout first"); }
}
```

Use of interfaces in this design 2024 Q3.4, 5m

- `DocumentState` is a **contract**: every state must implement every event, so nothing is forgotten.
- The context calls `state.submit(this)` without knowing which state it has: **polymorphism**, programming to an interface.
- New states plug in without changing `Document` (**Open/Closed**, **Dependency Inversion**).
- States are **interchangeable at runtime** and easy to mock in tests.

W5 Challenges → fix

- **Many classes** (one per state) → only use it when states are finite and behaviour really differs.
- **Transitions spread across classes** → keep the state diagram and table as documentation; validate in `setState()`.
- **A new event changes every state** → abstract base state with default "invalid action" methods.
- **Many state objects** → share stateless states as Singletons.

Mistakes that lose marks: the context still has `if (state == …)` · state diagram without a start dot or event labels · no `setState()` in the context · association instead of realisation from the states to the interface · transitions to states that aren't in the scenario.

10 SOLID principles

Five rules for designing classes that are easy to change. Lecture 12, new this year, so it may appear.

PRINCIPLE	IN PLAIN WORDS	ASK YOURSELF	PATTERNS THAT FOLLOW IT
Single Responsibility	A class should have only **one reason to change**.	Can I describe the class without "and"?	Facade (work stays in the subsystems), Decorator (one extra each), State (one class per state), Chain (one check per handler)
Open / Closed	Open for extension, **closed for modification**.	Can I add behaviour without editing tested code?	Decorator, Factory, Template, State, Chain, Adapter, Strategy: all add new classes instead of editing old ones
Liskov Substitution	A subclass must work **anywhere the parent works**, without surprises.	Can I swap in the child safely?	Template subclasses, State classes, Proxy, Decorator: all must be safe substitutes
Interface Segregation	Don't force a class to implement **methods it doesn't use**. Prefer small interfaces.	Is any class stuck with empty or throwing methods?	Adapter (narrow Target), role interfaces like `Workable`, `Eatable`
Dependency Inversion	High-level code should depend on **abstractions**, not concrete classes.	Does business logic `new` a concrete class?	Factory (returns interfaces), Facade / Adapter / Proxy behind interfaces, Dependency Injection (Spring)

S: Single Responsibility

```
// BAD: Order changes for 4 different reasons
public class Order {
  double calculateTotal() { ... } // rules
  void saveToDatabase() { ... }  // storage
  void printInvoice() { ... }    // display
  void sendEmail() { ... }       // messaging
}
```

```
// GOOD: one job each
class Order           // business rules
class OrderRepository // saves to the DB
class InvoicePrinter  // prints invoices
class EmailService    // sends emails
// OrderService (a facade) uses all four
```

Test: "If the database changes, should `Order` change?" If yes, responsibilities are mixed. Warning sign: "this class calculates orders **and** saves them **and** sends emails".

O: Open / Closed

```
// BAD: edit this for every new type
double discount(String t, double a) {
  if (t.equals("REGULAR")) return a * 0.05;
  if (t.equals("PREMIUM")) return a * 0.10;
  if (t.equals("VIP"))     return a * 0.20;
  return 0;   // STUDENT? edit again
}
```

```
// GOOD: add a class, change nothing
interface DiscountStrategy {
  double calculate(double amt);
}
class VipDiscount
      implements DiscountStrategy {
  public double calculate(double a) {
    return a * 0.20; }
}
// new StudentDiscount = new class only
```

Same idea for payments: a `PaymentMethod` interface, and `CryptoPayment` is added as a new class without touching the `PaymentProcessor`.

10 SOLID: L, I, D

And how to use SOLID to strengthen any pattern answer.

PRINCIPLES
L · I · D

L: Liskov Substitution

```
// VIOLATION: Square extends Rectangle
Rectangle r = new Square();
r.setWidth(5);
r.setHeight(10);
r.getArea();   // expected 50, got 100

// VIOLATION: Penguin extends Bird
class Penguin extends Bird {
  void fly() {
    throw new UnsupportedOperationException();
  }
}
```

```
// FIX: model the right abstraction
interface Shape { double getArea(); }
class Rectangle implements Shape { ... }
class Square    implements Shape { ... }

abstract class Bird { abstract void eat(); }
interface Flyable { void fly(); }
class Sparrow extends Bird
              implements Flyable { ... }
class Penguin extends Bird { ... }
```

Don't only ask "is it a kind of?" Ask "can the child keep every promise the parent makes?" No stricter inputs, no weaker results, no surprise exceptions.

I: Interface Segregation

```
// BAD: a fat interface
interface Worker {
  void work(); void eat(); void sleep();
}
// Robot must throw on eat() and sleep()

// GOOD: small role interfaces
interface Workable  { void work(); }
interface Eatable   { void eat(); }
interface Sleepable { void sleep(); }
class Human implements Workable,
    Eatable, Sleepable { ... }
class Robot implements Workable { ... }
```

Java's own examples: `Comparable`, `Runnable`, `Iterable`, `AutoCloseable`.

D: Dependency Inversion

```
// BAD: tied to MySQL, can't mock it
class UserService {
  private MySQLDatabase db =
      new MySQLDatabase();
}

// GOOD: depend on an interface,
// inject it through the constructor
class UserService {
  private final UserRepository repo;
  UserService(UserRepository r) {
    repo = r; }
}
new UserService(new PostgresUserRepo());
```

Dependency Inversion is the principle. **Dependency Injection** is the technique (pass dependencies in from outside; Spring does it for you).

Add one SOLID line to any pattern answer

cheap extra marks

Decorator, Factory, Strategy, State, Chain	"...which follows the **Open/Closed** principle: new types are new classes, existing code is not modified."
Facade, Chain handlers, Decorator	"...each class has a **single responsibility**."
Template subclasses, State, Proxy	"...any subclass can replace the parent (**Liskov**)."
Adapter Target, role interfaces	"...clients only see the small interface they need (**Interface Segregation**)."
Factory, any interface-based design	"...the client depends on an abstraction, not a concrete class (**Dependency Inversion**)."

SOLID vs patterns vs frameworks: SOLID are **principles** ("how should I design classes?"), patterns are **proven solutions** to recurring problems, frameworks are **reusable infrastructure** (Spring applies Dependency Inversion through injection). Using a pattern doesn't guarantee SOLID; it depends on how you apply it. And SOLID isn't "more interfaces everywhere": adding abstraction to a 2-line calculator is over-engineering (YAGNI).

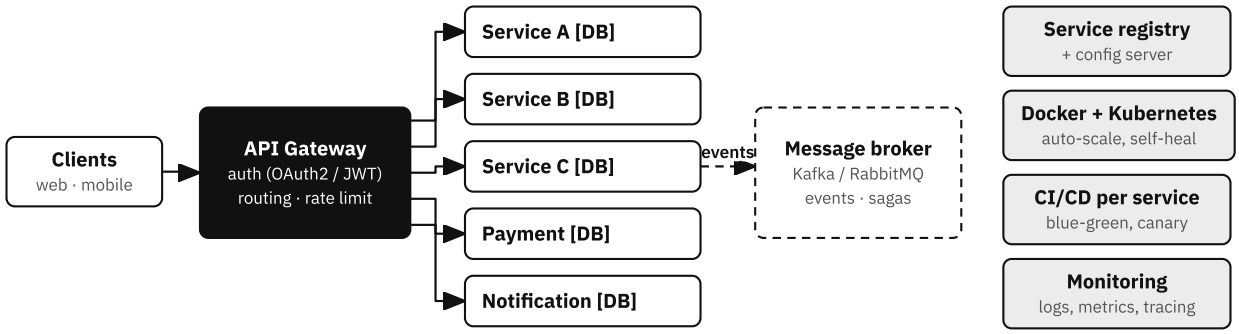
11 Microservices reference

A compact kit for an architecture question: diagram, scaling, architecture, consistency, security. Rename the services for the business in the question.

ARCHITECTURE
25 MARKS

2.1 Draw the scenario · 4–5m

copy this shape; rename Service A, B, C for your business



Services to use for each business

BUSINESS	SERVICES (PLUS PAYMENT / BILLING AND NOTIFICATION)
Food delivery	User · Restaurant & Menu · Order · Delivery / Rider tracking
Banking	Customer · Account · Transaction · Loan · Fraud detection
LMS	User · Course & Content · Enrolment · Assessment & Grading
Hospital	Patient · Appointment · Doctor & Staff · Pharmacy · Lab results
Ride-hailing	Rider · Driver · Trip matching · Pricing · Location
already asked	2023 supermarket chain: Purchasing, Sales/POS, Warehouse/Inventory, Supplier, Branch, Reporting · 2024 library: Book, User, Borrowing, Notification, Catalog Search (Elasticsearch) · 2025 streaming: User Profile, Content Delivery (transcode + CDN), Recommendation (ML), Billing/Subscription, Analytics

2.2 Scalability, fault isolation and continuous deployment 5m

Scalability	Each service **scales on its own**: more copies (pods) of only the busy service, with auto-scaling on CPU or request load. Example: scale Content Delivery and Search at peak time, not Billing. Keep services **stateless**, and use a cache / CDN.
Fault isolation	Each service runs in its **own process with its own database**, so one crash doesn't take down the rest. Use **circuit breakers**, timeouts, retries with back-off and bulkheads. **Degrade gracefully**: if Recommendations is down, show a "popular" list.
Continuous deployment	Each service has **its own repository and CI/CD pipeline**: automated tests, Docker image, then **blue-green or canary** release. APIs are versioned so one service can be updated without breaking others. One small team owns each service.
Manage it	Containers on **Kubernetes** (auto-scaling, self-healing), central **logging, metrics and tracing**, service registry and config server.

11 Microservices: the other parts

Architecture, data consistency and security. Each is worth 4–5 marks.

ARCHITECTURE
25 MARKS

2.3 Propose the architecture

4–5m

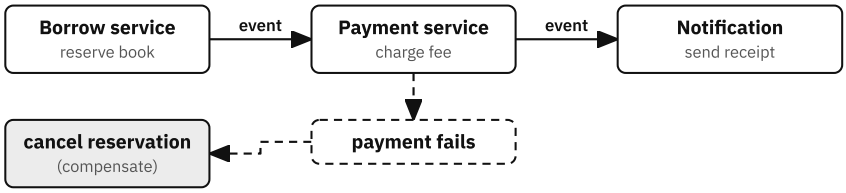
- **One service per business capability** (list yours from the table), each with its **own codebase, own database and own deployment**.
- **API Gateway** as the single entry point for web and mobile clients: routing, authentication, rate limiting.
- **Service registry** so services can find each other; **config server** for settings.
- Services talk **synchronously** with REST or gRPC when they need an answer now, and **asynchronously** with events on a **message broker** (Kafka / RabbitMQ) for everything else (e.g. "BookBorrowed" → Notification).
- Everything runs in **Docker containers on Kubernetes**, with central monitoring.

2.4 Keeping data consistent

4–5m

With a **database per service** there are no shared transactions, so we accept **eventual consistency** and use:

- **Saga pattern**: a business action is a series of local transactions. If one step fails, earlier steps are undone with **compensating actions** (payment fails → cancel the borrow / release the stock). It can be coordinated by events (choreography) or by one orchestrator service.
- **Event-driven updates**: a service publishes an event when its data changes; others update their own copy.
- **Transactional outbox** so the database change and the event are never out of step, and **idempotent consumers** so a repeated event does no harm.
- **CQRS** for read-heavy views (catalog search) built from events.
- Two-phase commit only where strong consistency is essential (rare, because it is slow and couples services).



2.5 Security

4–5m

Users	Login through **OAuth2 / OpenID Connect** at the gateway, which issues **JWT** tokens. **Role-based access** (RBAC): e.g. only librarians can add books.
Service to service	**Mutual TLS** (a service mesh such as Istio), short-lived tokens, and each service only allowed to call what it needs.
Data	HTTPS everywhere, encryption at rest, card data handled to PCI-DSS, secrets in a vault, each service's DB user has least privilege.
Attacks	Rate limiting, input validation, a web application firewall, audit logs and monitoring for unusual activity.

2023 Q2.6: "Why give every microservice platform the same hardware and configuration?" Identical environments (containers and infrastructure-as-code) make deployment, scaling and failover predictable: any node can run any service, performance is the same everywhere, capacity planning, automation and troubleshooting are simpler, and a failed node can be rebuilt exactly (DR). It avoids "works on my machine". Exception: GPU nodes for ML services.

12 Answer bank

Ready answers for the parts that can be attached to any scenario.

REUSABLE
ANSWERS

"How can you extend this design to achieve High Availability and Disaster Recovery?"

2022 Q3.5 · 2023 Q3.5, 5m

High Availability = the system keeps running when a part fails (little or no downtime).

- Run each component as several **stateless copies behind a load balancer**, with health checks and automatic restart (Kubernetes).
- **No single point of failure**: a Singleton or Facade on one server becomes replicas.
- **Database replication** / clusters with automatic failover.
- Timeouts, retries and **circuit breakers**; a **fallback handler** at the end of a Chain.

Disaster Recovery = getting back after a big failure (data centre lost).

- **Regular backups**, and the database **replicated to a second region**.
- Agreed **RPO** (how much data we can lose) and **RTO** (how fast we must be back).
- **Automated failover** to the DR site; infrastructure-as-code to rebuild quickly.
- **DR drills** to test it regularly.

Tie it to the pattern: "Because each filter / handler / service is stateless, extra copies can run on several servers..." makes the answer fit the question.

"Many parameters change over time. Improve configurability."

2023 Q1.5, 4m

Move every rate, limit and threshold out of the code into **external configuration** (a database table or a properties / YAML file). A **Singleton ConfigManager** loads it and every class reads from there. An admin screen changes values; changes are **versioned and audited** with an "effective from" date; **no recompiling or redeploying**. Example settings: `epf.employee=0.08`, `tax.threshold=100000`. (Code: tab 01 answers.)

Interface or abstract class?

"explain the use of interfaces / abstract classes"

	INTERFACE	ABSTRACT CLASS
Meaning	A contract: what a class **can do**	A partial parent: what a class **is**
Contains	Method signatures (plus default methods)	Fields, finished methods and abstract methods
Keyword	`implements`, a class can have many	`extends`, only one parent
Used in	Decorator Component, Adapter Target, Proxy Subject, State, Factory Product	Template Method base, Decorator base, Chain handler

Why they help (write this): the client depends on the **abstraction**, not the concrete class, so different implementations can be swapped through **polymorphism**. New classes plug in without changing the client (**Open/Closed**, **Dependency Inversion**), and tests can use mocks. An abstract class is chosen when shared code or a fixed order must live in the parent.

Pros and cons of using design patterns in general

- **+** Proven solutions, so fewer design mistakes
- **+** Shared vocabulary for the team
- **+** Loose coupling, reuse, flexibility
- **+** Easier to maintain and extend
- **−** More classes and more abstraction
- **−** Learning curve for new developers
- **−** Overuse is over-engineering
- **−** Ask: "is it worth the extra abstraction?"

12 Answer bank: other patterns

Patterns named in the lectures but not taught in detail, and general fallback answers.

REUSABLE
ANSWERS

If the scenario fits a pattern you weren't taught

name it, give one reason, sketch it

PATTERN	THE QUESTION WOULD SAY…	ONE-LINE ANSWER
Strategy behavioural	different discount rules / pricing / sorting, chosen at runtime	Each rule is a class behind one interface (`DiscountStrategy`); the context holds one and can swap it.
Observer behavioural	notify customers when the order status changes; subscribers	The subject keeps a list of observers and calls `update()` on all of them when it changes.
Command behavioural	undo and redo; queue or log requests	Each action is an object with `execute()` and `undo()`, kept on a history stack.
Builder creational	build a complex report with many optional sections, step by step	A builder adds parts one at a time (`addHeader().addChart()`) and `build()` returns the result.
Abstract Factory creational	create families of related objects that must match (a UI theme)	One factory interface creates every product of a family; one concrete factory per family.
Composite structural	tree structures: folders and files, menus inside menus	Leaf and container share one interface, so the client treats one item and a group the same way.

From Lecture 1's activity: multiple payment methods → Strategy / Factory · user account types → Factory · order updates → Observer · new payment gateway → Adapter · complex report → Builder · discount rules → Strategy · simple access to a library system → Facade · one configuration manager → Singleton · undo/redo → Command · order status → State · coffee extras → Decorator · requests through several officers → Chain.

"Why not just use inheritance for everything?"

Inheritance fixes behaviour at compile time. Every combination needs its own subclass (**class explosion**), the hierarchy becomes rigid, and a change in the parent ripples into every child. So prefer **composition**: Decorator, State, Strategy and Adapter all hold another object instead of inheriting from it, and can change at runtime.

Challenges that fit almost any pattern design

use when the pattern's own W5 isn't enough

CHALLENGE	MECHANISM TO OVERCOME IT
More classes, more complex to understand	Clear naming, a class diagram in the documentation, only use the pattern where change is likely.
Harder to debug (calls pass through layers)	Logging at each layer, unit tests for each class.
Performance overhead	Usually negligible; cache results; keep chains and wrappers short.
Single point of failure (Singleton, Facade, Proxy)	Stateless replicas behind a load balancer; standby copy.
Third-party parts change	Keep them behind an interface or Adapter so only one class changes.
Values change often	External configuration read by a Singleton ConfigManager.

13 Worked answer: 2025 Q4

A full multi-pattern question, answered at exam length. This is the style to expect.

WORKED
25 MARKS

The question: an e-commerce platform. Credit card, PayPal and cash-on-delivery payments follow a common workflow (validate → process → send receipt), but processing differs. Orders go through automated checks (inventory, fraud, delivery area) that must be maintainable and extendable. A single global OrderID generator must give every order a unique ID.

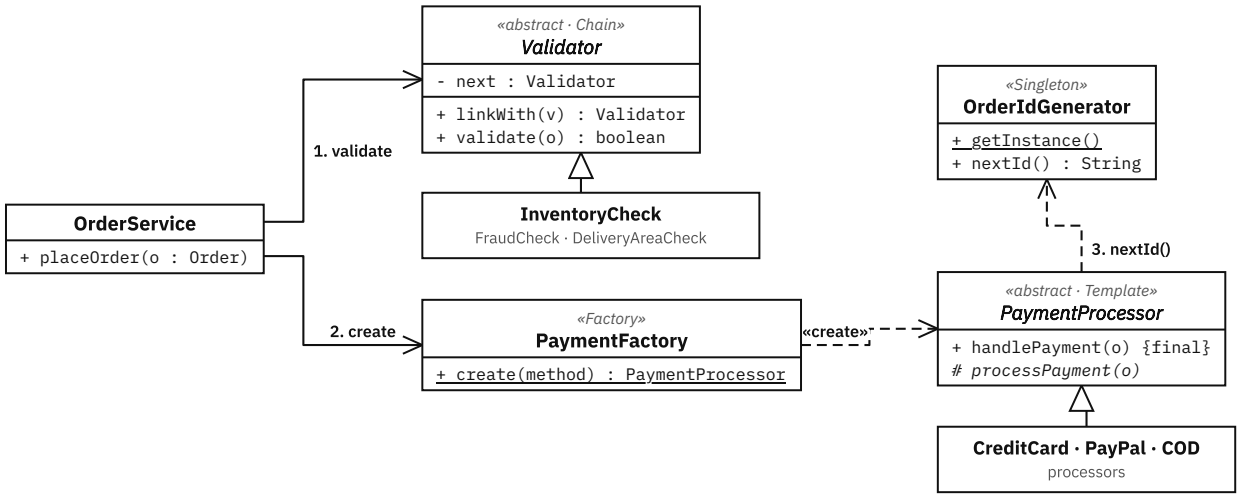
4.1 Which patterns fit? Give reasons.

8m, 2 marks each

CASE	PATTERN	REASON (ONE OR TWO LINES IS ENOUGH)
(a) common payment steps, different processing	**Template Method**	The order of steps is fixed in a final `handlePayment()`; only `processPayment()` is written by each payment type, so the workflow is standard and nothing is duplicated.
(b) checks in sequence, easy to extend	**Chain of Responsibility**	Each check is a handler linked to the next; a failure stops the order. A new check is one new class and one link, with no change to the others.
(c) one global OrderID generator	**Singleton**	Only one generator exists, so one counter is shared by every part of the system and IDs are never duplicated.
(d) creating payment objects	**Factory Method**	The customer picks the method at runtime; `PaymentFactory.create(method)` returns the right processor, so the client has no `new` or if-else.

How the four fit together

draw this if they ask for one diagram of the whole design



The rest of the question

where to copy from

4.2 abstract PaymentProcessor (4m)	tab 07 W3, left code
3.3 CreditCardPaymentProcessor (3m)	tab 07 W3, right code
3.4 validation flow (5m)	tab 08 answers, Validator code
3.5 OrderID generator (5m)	tab 01 answers, OrderIdGenerator

```
public void placeOrder(Order o) {
  Validator chain = new InventoryCheck();
  chain.linkWith(new FraudCheck())
       .linkWith(new DeliveryAreaCheck());
  if (!chain.validate(o)) return; // Chain
  o.setId(OrderIdGenerator.getInstance()
          .nextId());           // Singleton
  PaymentProcessor p = PaymentFactory
      .create(o.getMethod());   // Factory
  p.handlePayment(o);           // Template
}
```

13 Worked answer: 2025 Q1

The self-checkout kiosk: State + Decorator + Singleton in one scenario.

WORKED
25 MARKS

The question: a supermarket self-checkout kiosk. Customers scan items, add optional extras (gift-wrap, eco-bag, express packing) and pay. The kiosk works in states: Idle, Scanning, Payment, Completed. Item prices change with add-ons without a subclass for every combination. One global tax calculator must be used by every checkout counter.

1.1 Which patterns, with reasons

6m, 2 each

(a) kiosk states	**State**	The kiosk behaves differently in each state (pay is refused while Idle). Each state is a class that handles the actions and the next transition, so there is no if/else on a status field.
(b) optional add-ons, dynamic price	**Decorator**	Each add-on wraps the item and adds its own cost and description, so any combination is built at runtime without 2^n subclasses.
(c) one tax calculator	**Singleton**	One instance shared by all counters means one rate and one rule set, so tax is consistent everywhere.

1.2 + 1.3 Interface and class

5m + 5m

```
public interface CheckoutItem {
  double getCost();
  String getDescription();
}
public class GroceryItem
        implements CheckoutItem {
  private String name;
  private double price;
  public GroceryItem(String n, double p) {
    name = n; price = p; }
  public double getCost() {
    return price; }
  public String getDescription() {
    return name; }
}
```

Add-on decorators (GiftWrap etc.): tab 06 answers.

1.4 Kiosk states, example transition

5m

`Kiosk` (context) holds a `KioskState`; `IdleState`, `ScanningState`, `PaymentState`, `CompletedState` implement `scanItem()`, `checkout()`, `pay()`.

Example: in Scanning, pressing Checkout adds up the items, adds tax and calls `kiosk.setState(new PaymentState())`. State diagram and code: tab 09 answers.

1.5 Singleton in the pricing flow

4m

Scanned items are decorated with their add-ons, so `item.getCost()` is the final item price. In the Payment state the kiosk adds them up and calls `TaxCalculator.getInstance().calculateTax(subtotal)`. Every counter gets the same instance, so a rate change applies everywhere at once.

What these two worked answers teach

- Every sub-part names **one pattern and one or two reasons tied to the scenario**. Short is fine.
- The code parts are **small**: an interface, one class, or one method. Use the names in the question.
- The later parts ask how the patterns **connect** ("integrate the Singleton into the pricing flow"). Say which class calls which, in which state or step.